

BK Restaurant Management System

System Design Project Final Report

Project Team Members

Sachin Lakshitha (Student ID: SE/2022/020)

Duvini Nimethra (Student ID: SE/2022/023)

Dinuja Ranaweera (Student ID: SE/2022/026)

Nethmi Navodya (Student ID: SE/2022/030)

Supervisor: Dr. Tiroshan Madushanka

FACULTY OF SCIENCE

Software Engineering Teaching Unit

University of Kelaniya, Sri Lanka

SENG 31242 – System Design Project

FINAL DESIGN REPORT

BK Restaurant Management System

Project: BK Restaurant Management System

GitHub Organisation: bk-restaurant-system

Repository: bk-restaurant-management-system

Programme: B.Sc. (Hons.) in Software Engineering

Year / Semester: Year 3, Semester 1

Course Code: SENG 31242

Date: 2026

Version: 2.0

Declaration

We, the undersigned, hereby declare that this report is our own original work and has not been submitted, in whole or in part, for any other degree or qualification. All sources consulted have been duly acknowledged.

Sachin Lakshitha (SE/2022/020)

Signature: _____ Date: _____

Duvini Nimethra (SE/2022/023)

Signature: _____ Date: _____

Dinuja Ranaweera (SE/2022/026)

Signature: _____ Date: _____

Nethmi Navodya (SE/2022/030)

Signature: _____ Date: _____

Supervisor Declaration

I confirm that I have supervised this project and that the work submitted is satisfactory for the purposes of examination.

Supervisor Name: Dr. Tiroshan Madushanka Date: _____

Signature: _____

Acknowledgements

We would like to express our deepest gratitude to our academic supervisors and the Software Engineering Teaching Unit at the University of Kelaniya for their invaluable guidance. We also extend our sincere thanks to Mr. N. C. Sanjeewa De Silva, the owner of BK Restaurant & BK Bar, for providing us with the opportunity to analyze and design a system for his business, and for his continuous feedback throughout the project lifecycle.

Abstract

The operational workflow across multiple restaurant branches is digitized and centralized by the proposed BK Restaurant Management System. Inefficient manual processes are replaced with an integrated digital platform. Currently, ad-hoc WhatsApp orders, verbal kitchen communication, and physical ledger books for inventory are relied upon by the restaurant, leading to operational errors and a lack of centralized oversight. A comprehensive n-tier architecture is introduced by the proposed system, featuring a web-based Point of Sale (POS), an automated Kitchen Display System (KDS), a Flutter-based mobile application for GPS delivery tracking, and a centralized management dashboard. Over an 8-week period, rigorous fact-finding, requirements engineering, and architectural design phases were executed. Sourced from the Software Requirements Specification (SRS) and Software Design Specification (SDS), a scalable, reliable, and cost-effective

solution utilizing modern web technologies including React, Node.js, and PostgreSQL is provided by the resulting design.

Keywords: System Design, POS, KDS, N-Tier Architecture, React, Node.js, Flutter, Restaurant Management

Table of Contents

1. Introduction
2. System Analysis
3. System Design
4. Conclusion
5. References

List of Figures

1. Figure 1.1: Project Timeline (Gantt Chart)
2. Figure 2.1: Context Diagram
3. Figure 2.2: Use Case Diagram
4. Figure 2.3: Activity Diagram - UC-01
5. Figure 2.4: Activity Diagram - UC-04
6. Figure 2.5: Activity Diagram - UC-05
7. Figure 2.6: Activity Diagram - UC-07
8. Figure 2.7: Activity Diagram - UC-09
9. Figure 2.8: Activity Diagram - UC-10
10. Figure 2.9: Sequence Diagram - UC-01
11. Figure 2.10: Sequence Diagram - UC-02

12. Figure 2.11: Sequence Diagram - UC-03
13. Figure 2.12: Sequence Diagram - UC-04
14. Figure 2.13: Sequence Diagram - UC-06
15. Figure 2.14: Sequence Diagram - UC-08
16. Figure 2.15: Sequence Diagram - UC-09
17. Figure 2.16: Sequence Diagram - UC-12
18. Figure 2.17: Sequence Diagram - UC-14
19. Figure 2.18: Sequence Diagram - UC-15
20. Figure 3.1: Component Diagram
21. Figure 3.2: Deployment Diagram
22. Figure 3.3: Class Diagram / ERD State Behaviors
23. Figure 3.4: Sequence Diagram - UC-02 (POS Order Entry)
24. Figure 3.5: Sequence Diagram - UC-03 (Modify/Cancel Order)
25. Figure 3.6: Sequence Diagram - UC-04 (Kitchen Order Processing)
26. Figure 3.7: Sequence Diagram - UC-07 (Rider Assignment)
27. Figure 3.8: Sequence Diagram - UC-09 (Live Delivery Tracking)
28. Figure 3.9: Sequence Diagram - UC-10 (Inventory Adjustment)
29. Figure 3.10: Use Case Diagram (Role-Based Access)
30. Figure 3.11: Customer Order Creation Flow
31. Figure 3.12: Kitchen Order Processing Flow
32. Figure 3.13: Delivery Rider Flow

- 33. Figure 3.14: Inventory Management Flow
- 34. Figure 3.15: Owner Monitoring Flow
- 35. Figure 3.16: UI - Staff Registration
- 36. Figure 3.17: UI - Employee Dashboard
- 37. Figure 3.18: UI - Manager Dashboard (Global)
- 38. Figure 3.19: UI - Overall Analytics
- 39. Figure 3.20: UI - Inventory Dashboard
- 40. Figure 3.21: UI - Login Screen

List of Tables

- 1. Table 2.1: User Classes and Characteristics
- 2. Table 2.2: Operating Environment
- 3. Table 2.3: Fact-Gathering Techniques
- 4. Table 2.4: Identified Pain Points and Solutions
- 5. Table 2.5: Functional Requirements Summary
- 6. Table 2.6: Non-Functional Requirements Summary

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
ADR	Architectural Decision Record
AWS	Amazon Web Services
DB	Database

ERD	Entity-Relationship Diagram
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token – used for secure session authentication.
KDS	Kitchen Display System – a screen in the kitchen showing incoming orders.
NoSQL	Not Only SQL
ODM	Object Document Mapper
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
POS	Point of Sale
RBAC	Role-Based Access Control
REST API	Representational State Transfer Application Programming Interface.
SaaS	Software as a Service
SDS	System Design Specification
SLA	Service Level Agreement
SME	Small and Medium Enterprise.
SPA	Single Page Application
SQL	Structured Query Language

SRS	Software Requirements Specification
UC	Use Case – a description of system interactions with actors.
UML	Unified Modeling Language
UI	User Interface
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Background

Efficient order processing, accurate kitchen communication, and reliable delivery logistics are heavily relied upon within the hospitality and restaurant industry. A predominantly manual and fragmented workflow is currently utilized by BK Restaurant and BK Bar, a growing enterprise with branches in Ambalangoda and Kahawa. The agility required to scale and meet modern customer expectations is found to be lacking in this traditional approach to restaurant management. To bridge this gap, modern enterprise systems require centralized digital control to coordinate real-time transaction updates, resource utilization, and client-facing communication.

The primary operations of BK Restaurant consist of handling walk-in dining customers, executing takeaway orders, and managing self-operated delivery fleets. Due to the lack of an integrated technical infrastructure, these operations are conducted independently at each branch. Information silos are created, and significant operational latencies are introduced. By establishing a unified system design, these disparate workflows can be consolidated into a centralized platform.

1.2 Problem

Several critical pain points are presented by the current manual system. Customer orders are primarily received via ad-hoc WhatsApp messages and phone calls, which must be manually processed by cashiers. Once taken, orders are verbally relayed to the kitchen staff, which frequently results in preparation errors and missed modifications. Furthermore, delivery logistics are left unmonitored; once the premises are left by a rider, no visibility

into their location or delivery status is maintained by the branch manager. Finally, inventory is managed using physical ledger books, whereby real-time, centralized oversight of stock levels, sales, and employee attendance across the three branches cannot be maintained by the business owner.

In addition to day-to-day transaction errors, the manual inventory tracking system leads to frequent stock-outs of critical raw materials. Since stock counts are conducted manually at the end of each week, shortages are often only identified when items are completely depleted. Financial performance is also negatively impacted because the business owner is unable to perform real-time audit controls on cash collections and sales summaries, exposing the enterprise to potential losses.

1.3 Aims

The construction of a unified, digital ecosystem that centralizes branch operations is primarily aimed for by the BK Restaurant Management System. Sourced from the requirements and design specifications, the following objectives are established:

Walk-in and external order workflows are to be digitized through a robust Point of Sale (POS) interface.

Verbal miscommunications are to be eliminated by the implementation of a synchronized Kitchen Display System (KDS) refreshing within 2 seconds.

Delivery logistics and tracking are to be automated via a proprietary Rider App with live GPS integration.

A cloud-synced, multi-branch dashboard is to be provided for the owner, enabling real-time attendance auditing, leave management, and sales analytics.

Stock-outs are to be mitigated by transitioning to a real-time digital tracking system with automated low-stock alerts and direct digital supplier requests.

1.4 Scope

The design and requirements specification of the core management platform are encompassed within the scope of this project, as detailed in the Software Requirements Specification (SRS) in Appendix A. The POS, KDS, Rider App, and the centralized

backend APIs required to support role-based access control (RBAC) are included. The scope is extended to inventory stock-out alerts and digital supplier requests, as well as digital leave management. Hardware installation and third-party food delivery app integrations (e.g., UberEats, PickMe) are considered out of scope for the initial phase.

1.5 Organisation of Report

This report is organized into six main chapters. The Introduction is provided in Chapter 1. The Project Methodology is defined in Chapter 2. The System Analysis is detailed in Chapter 3, where fact-finding methods, use case modeling, and requirements engineering are covered. The System Design is presented in Chapter 4, detailing architectural patterns, UML modeling, and database schemas. The Conclusion is provided in Chapter 4. The full SRS and SDS documents are attached as appendices to serve as exhaustive technical sources.

1.6 Project Timeline

The project followed an 8-week agile timeline, tracking the progression from idea discovery to the final system design. The schedule was managed via GitHub Projects, with a breakdown of major milestones illustrated in the Gantt chart below.

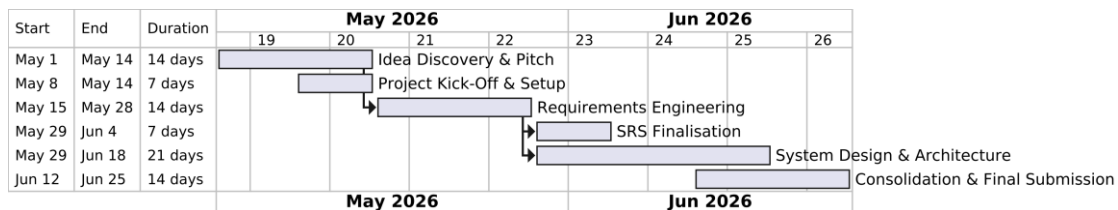


Figure 1.1: Project Timeline Gantt Chart

CHAPTER 2: SYSTEM ANALYSIS

2.1 Fact-Finding Documentation

Rigorous fact-finding was conducted to accurately capture the operational requirements of BK Restaurant. A comprehensive mixed-methods approach was employed, combining structured stakeholder interviews with direct on-site observations. This dual approach ensured that both managerial expectations and ground-level operational bottlenecks were accurately identified. For instance, detailed interviews with the proprietor, Mr. Sanjeewa,

revealed critical issues with WhatsApp-based order ingestion and a severe lack of centralized oversight. Direct observation confirmed these claims, noting significant manual transcription delays. Two formal techniques were employed and documented:

Technique	Description	Artefacts Produced
Client Interviews	Structured interviews were conducted with Mr. N. C. Sanjeewa De Silva to understand current workflows and desired outcomes.	Interview transcripts, pain-point list.
On-Site Observation	Direct observation of the kitchen and cashier workflows was executed during peak hours.	Process notes, timing data, paper forms.

2.2 Existing System Analysis & Pain Points

The critical pain points of the manual system were identified and mapped to proposed system resolutions.

ID	Pain Point	Impact	Proposed Resolution
P-01	Ad-hoc WhatsApp orders are manually transcribed.	Preparation errors and delayed orders.	A unified digital POS interface is implemented.
P-02	Kitchen communication relies on verbal shouting.	Missed modifiers and high stress.	An automated Kitchen Display System (KDS) is deployed.
P-03	Delivery riders cannot be tracked once dispatched.	Lack of visibility for the manager.	GPS tracking via a Flutter-based Rider App is integrated.
P-04	Inventory is managed via physical ledger books.	Frequent stock-outs and cash leakage.	Real-time digital inventory tracking is established.

2.3 User Classes and Characteristics

User profiles were synthesized from the fact-finding phase to guide system design.

Persona	Demographics & Tech Level	Responsibilities & Pain Points
Sanjeewa (Owner)	45 yrs, Moderate Tech	Needs multi-branch oversight. Pain point: Cash leakage and inventory blind spots.
Kasun (Manager)	32 yrs, High Tech	Handles local stock and shifts. Pain point: Surprised by stock-outs mid-shift.
Nimali (Cashier)	22 yrs, High Tech	Inputs orders under high pressure. Pain point: WhatsApp order transcription errors.
Ruwan (Rider)	26 yrs, Moderate Tech	Outdoor delivery. Pain point: No structured routing, reliant on phone calls.

2.4 Functional Requirements

The core functional requirements (FRs) were rigorously mapped to their source and verification methods.

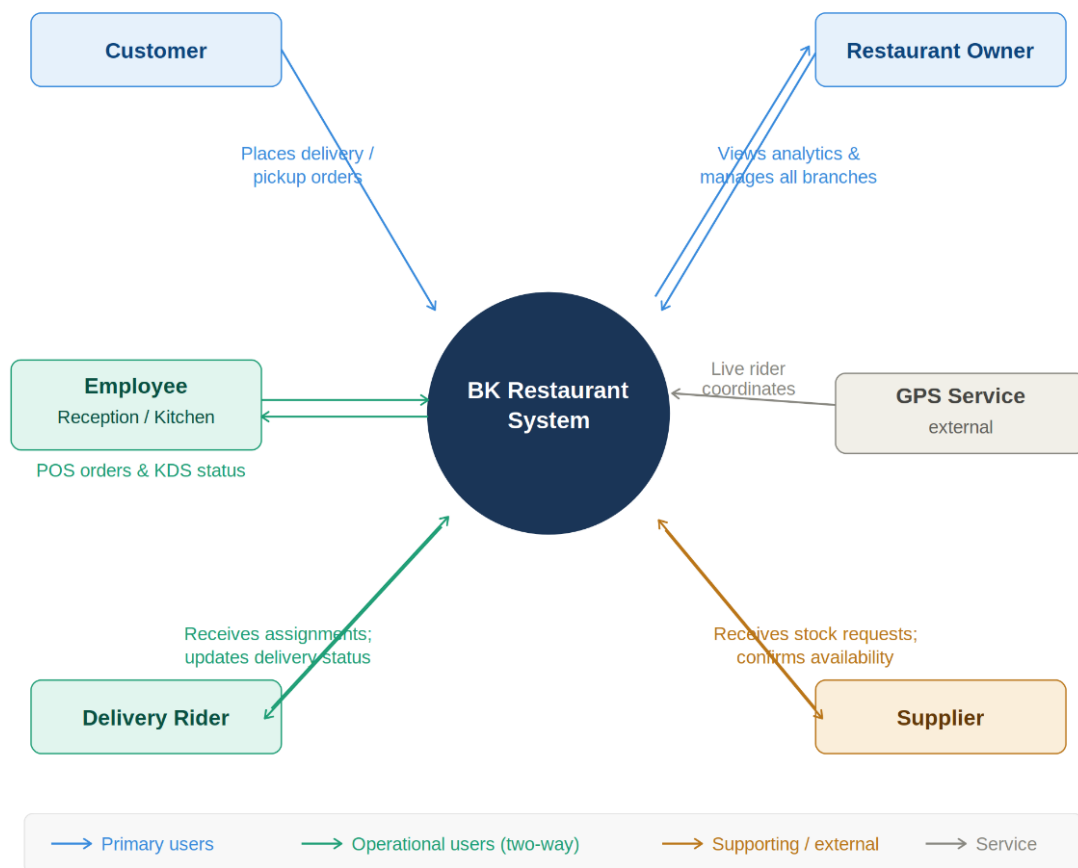
ID	Requirement (Title & Description)	Priority	Source	Test
FR-01	Central Dashboard: A unified dashboard for multi-branch tracking must be provided.	High	Client Interview	TC-FR-01
FR-02	POS Order Entry: Orders must be permitted to be input swiftly by cashiers.	High	Observation	TC-FR-02
FR-03	Kitchen Display Sync: The KDS is	High	Survey	TC-FR-

	required to update within 2s of order placement.			03
FR-04	Rider GPS Tracking: Real-time tracking of assigned delivery riders must be enabled.	Medium	Client Interview	TC-FR-04
FR-05	Inventory Alerts: Automated low-stock notifications must be transmitted to managers.	Medium	Competitor Analysis	TC-FR-05

2.5 System Boundary Diagrams

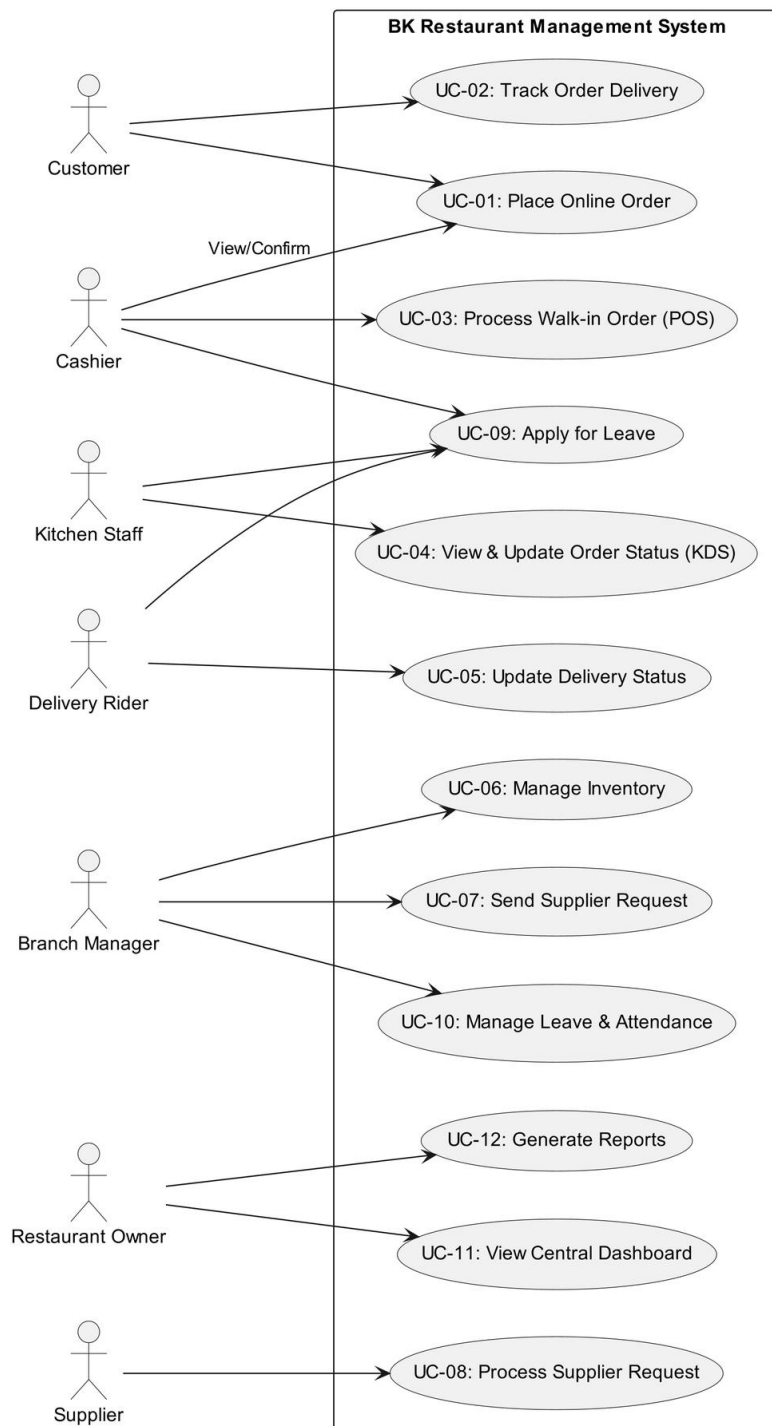
The overall system boundaries and external entity interactions were modeled using Context and Use Case diagrams.

Figure 2.1: Context Diagram



Context Diagram

Figure 2.2: Use Case Diagram



Use Case Diagram

2.6 Use Cases

The complete scope of system interactions was modeled through detailed use cases. Explicit business rules were embedded within each use case.

UC-01: Log In and Access Role-Based Features

Field	Content
Use Case ID	UC-01
Use Case Name	Log In and Access Role-Based Features
Actor(s)	Restaurant Owner, Branch Managers, Employees/Cashiers, Delivery Riders, Suppliers
Preconditions	User is registered in the system and is not currently logged in.
Postconditions Success	– User is authenticated and redirected to their role-specific dashboard.
Postconditions Failure	– Authentication fails; user remains on the login page.
Main Flow	1. User navigates to the login screen. 2. User enters username and password. 3. System validates credentials. 4. System retrieves user role. 5. System grants access to role-based features.
Alternative Flow 1	At step 3: If credentials are invalid, system displays an 'Invalid login' error message.
Alternative Flow 2	At step 2: User clicks 'Forgot Password', redirecting to password reset flow.
Business Rules	BR-01: Users can only access modules permitted by their assigned role.

UC-02: Create Customer Orders

Field	Content
Use Case ID	UC-02
Use Case Name	Create Customer Orders
Actor(s)	Employees/Cashiers
Preconditions	Employee is logged in to the POS module.
Postconditions – Success	Order is recorded, inventory is marked for update, and kitchen is notified.
Postconditions – Failure	Order is discarded; no inventory changes occur.
Main Flow	1. Employee initiates new order. 2. Employee selects order type (Dine-in, Takeaway, Delivery). 3. Employee adds menu items. 4. System calculates total. 5. Employee confirms order and accepts payment. 6. System saves order.
Alternative Flow 1	At step 3: If an item is out of stock, system prevents adding it and displays an alert.
Alternative Flow 2	None
Business Rules	BR-02: Order total must be greater than zero.

UC-03: Modify and Cancel Orders

Field	Content
Use Case ID	UC-03
Use Case Name	Modify and Cancel Orders
Actor(s)	Employees/Cashiers
Preconditions	Employee is logged in; an active order exists.

Postconditions Success	–	Order is successfully updated or canceled; system logs the change.
Postconditions Failure	–	Order remains in its previous state.
Main Flow		1. Employee searches for an active order. 2. System displays order details. 3. Employee selects 'Modify' or 'Cancel'. 4. Employee makes changes or confirms cancellation. 5. System updates order status and recalculates totals if modified.
Alternative Flow 1		At step 3: If order is already in 'Dispatched' or 'Completed' status, system denies modification.
Alternative Flow 2		None
Business Rules		BR-03: Orders cannot be canceled once they are dispatched.

UC-04: Update Order Status

Field		Content
Use Case ID		UC-04
Use Case Name		Update Order Status
Actor(s)		Employees/Cashiers
Preconditions		Employee is logged in; active orders exist in the queue.
Postconditions Success	–	Order status is updated (e.g., Preparing, Ready, Completed).
Postconditions Failure	–	Status update fails; error logged.
Main Flow		1. Employee views active order queue. 2. Employee selects an order. 3. Employee changes status from 'Pending' to 'Preparing'. 4. System saves the new status. 5. Status updates on real-time

	displays.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-04: Status progression must follow a logical sequence (Pending -> Preparing -> Ready).

UC-05: Generate Bills and Invoices

Field	Content
Use Case ID	UC-05
Use Case Name	Generate Bills and Invoices
Actor(s)	Employees/Cashiers
Preconditions	Order has been created and confirmed.
Postconditions Success	– Digital bill/invoice is generated and stored in order history.
Postconditions Failure	– Invoice generation fails; user is prompted to retry.
Main Flow	1. Employee finalizes an order. 2. System calculates final amount including taxes/fees. 3. System generates a digital invoice. 4. System offers option to print or email the invoice. 5. System archives invoice in order history.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-05: Invoices must include a unique transaction ID and timestamp.

UC-06: Place External Orders

Field	Content
Use Case ID	UC-06
Use Case Name	Place External Orders
Actor(s)	Customers
Preconditions	Customer has access to phone or WhatsApp.
Postconditions – Success	Order is received by the restaurant and entered into the system.
Postconditions – Failure	Order is not placed (e.g., outside delivery hours).
Main Flow	1. Customer contacts restaurant via Phone/WhatsApp. 2. Customer provides order details and delivery address. 3. Employee enters details into the system (invoking UC-02). 4. Customer receives confirmation.
Alternative Flow 1	At step 2: If address is outside delivery zone, customer is informed.
Alternative Flow 2	None
Business Rules	BR-06: Delivery orders require a valid contact number and address.

UC-07: Receive Delivery Assignments

Field	Content
Use Case ID	UC-07
Use Case Name	Receive Delivery Assignments
Actor(s)	Delivery Riders

Preconditions	Rider is logged into the mobile app and marked as 'Available'.
Postconditions Success	– Order is assigned to the rider; order status becomes 'Dispatched'.
Postconditions Failure	– Assignment fails; order remains 'Ready for Delivery'.
Main Flow	1. System identifies a 'Ready' delivery order. 2. System assigns order to an available rider. 3. Rider receives notification on mobile app. 4. Rider accepts assignment. 5. System updates order status to 'Dispatched'.
Alternative Flow 1	At step 4: Rider rejects assignment; system reassigns to the next available rider.
Alternative Flow 2	None
Business Rules	BR-07: Riders can only be assigned orders from their designated branch.

UC-08: Update Delivery Status via Mobile

Field	Content
Use Case ID	UC-08
Use Case Name	Update Delivery Status via Mobile
Actor(s)	Delivery Riders
Preconditions	Rider has an active assigned delivery.
Postconditions Success	– Delivery is marked 'Completed'; customer notified.
Postconditions Failure	– Status remains unchanged; rider prompted to retry.

Main Flow	1. Rider arrives at delivery location. 2. Rider hands over the order. 3. Rider opens mobile app. 4. Rider marks order as 'Delivered'. 5. System logs completion time and updates central database.
Alternative Flow 1	At step 2: Customer is unavailable; rider marks order as 'Delivery Failed'.
Alternative Flow 2	None
Business Rules	BR-08: Riders must be within reasonable GPS proximity to the delivery address to mark as delivered.

UC-09: Monitor Delivery Progress

Field	Content
Use Case ID	UC-09
Use Case Name	Monitor Delivery Progress
Actor(s)	Restaurant Owner
Preconditions	Owner is logged in; deliveries are currently active.
Postconditions Success	– Owner successfully views real-time delivery locations.
Postconditions Failure	– Map or tracking data fails to load.
Main Flow	1. Owner navigates to 'Delivery Tracking' dashboard. 2. System retrieves GPS coordinates of active riders. 3. System overlays rider locations and estimated delivery times on a map view. 4. Owner selects a specific rider for detailed order info.
Alternative Flow 1	At step 2: If a rider's GPS is off, system displays their last known location and a warning.

Alternative Flow 2	None
Business Rules	BR-09: GPS data must be updated at least every 60 seconds.

UC-10: Manage Local Inventory

Field	Content
Use Case ID	UC-10
Use Case Name	Manage Local Inventory
Actor(s)	Branch Managers
Preconditions	Branch Manager is logged into the inventory module.
Postconditions Success	– Inventory counts are accurately adjusted and saved.
Postconditions Failure	– Inventory update fails; previous counts remain.
Main Flow	1. Manager navigates to 'Inventory Management'. 2. System displays current stock levels. 3. Manager performs manual stock count. 4. Manager enters adjustments into the system. 5. System saves updated inventory records.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-10: Inventory updates automatically after sales; manual adjustments require a logged reason.

UC-11: Review Low-Stock Alerts

Field	Content
Use Case ID	UC-11

Use Case Name	Review Low-Stock Alerts
Actor(s)	Branch Managers
Preconditions	Inventory item falls below the defined threshold.
Postconditions Success	– Manager acknowledges alert and initiates a stock request.
Postconditions Failure	– Manager ignores alert; item risks stocking out.
Main Flow	1. System detects an item count below minimum threshold. 2. System generates a low-stock alert. 3. Manager reviews alerts on dashboard. 4. Manager acknowledges the alert. 5. Manager proceeds to create a stock request.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-11: Alerts remain active until inventory count is increased above the threshold.

UC-12: Process Stock Requests

Field	Content
Use Case ID	UC-12
Use Case Name	Process Stock Requests
Actor(s)	Branch Managers, Suppliers
Preconditions	Manager has identified a need for supplies.
Postconditions Success	– Stock request is submitted and fulfilled by supplier.

Postconditions Failure	–	Stock request is rejected or fails to send.
Main Flow		1. Manager creates a new stock request. 2. Manager adds items and quantities. 3. System sends request to corresponding Supplier. 4. Supplier logs in and reviews request. 5. Supplier approves and dispatches items. 6. Manager confirms receipt.
Alternative Flow 1		At step 5: Supplier lacks stock and rejects/modifies the request.
Alternative Flow 2		None
Business Rules		BR-12: Received stock automatically updates branch inventory counts.

UC-13: Manage Employee Shifts and Schedules

Field		Content
Use Case ID		UC-13
Use Case Name		Manage Employee Shifts and Schedules
Actor(s)		Branch Managers
Preconditions		Manager is logged into the employee module.
Postconditions Success	–	Weekly schedule is published and accessible to employees.
Postconditions Failure	–	Schedule is not saved.
Main Flow		1. Manager opens the scheduling tool. 2. Manager assigns employees to specific shifts. 3. System checks for scheduling conflicts. 4. Manager finalizes the schedule. 5. System notifies employees of their upcoming shifts.

Alternative Flow 1	At step 3: System detects an overlapping shift and prompts manager to resolve it.
Alternative Flow 2	None
Business Rules	BR-13: Employees cannot be scheduled for overlapping shifts.

UC-14: Record Employee Attendance

Field	Content
Use Case ID	UC-14
Use Case Name	Record Employee Attendance
Actor(s)	Employees/Cashiers
Preconditions	Employee is physically at the branch and has system access.
Postconditions Success	– Clock-in/clock-out time is recorded in the system.
Postconditions Failure	– Attendance record fails to save.
Main Flow	1. Employee accesses the attendance module. 2. Employee enters unique pin or credentials. 3. Employee selects 'Clock In' (or 'Clock Out'). 4. System logs the timestamp and employee ID. 5. System displays confirmation.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-14: Clock-in must occur within the designated branch network or location.

UC-15: Monitor Centralized Branch Operations & Analytics

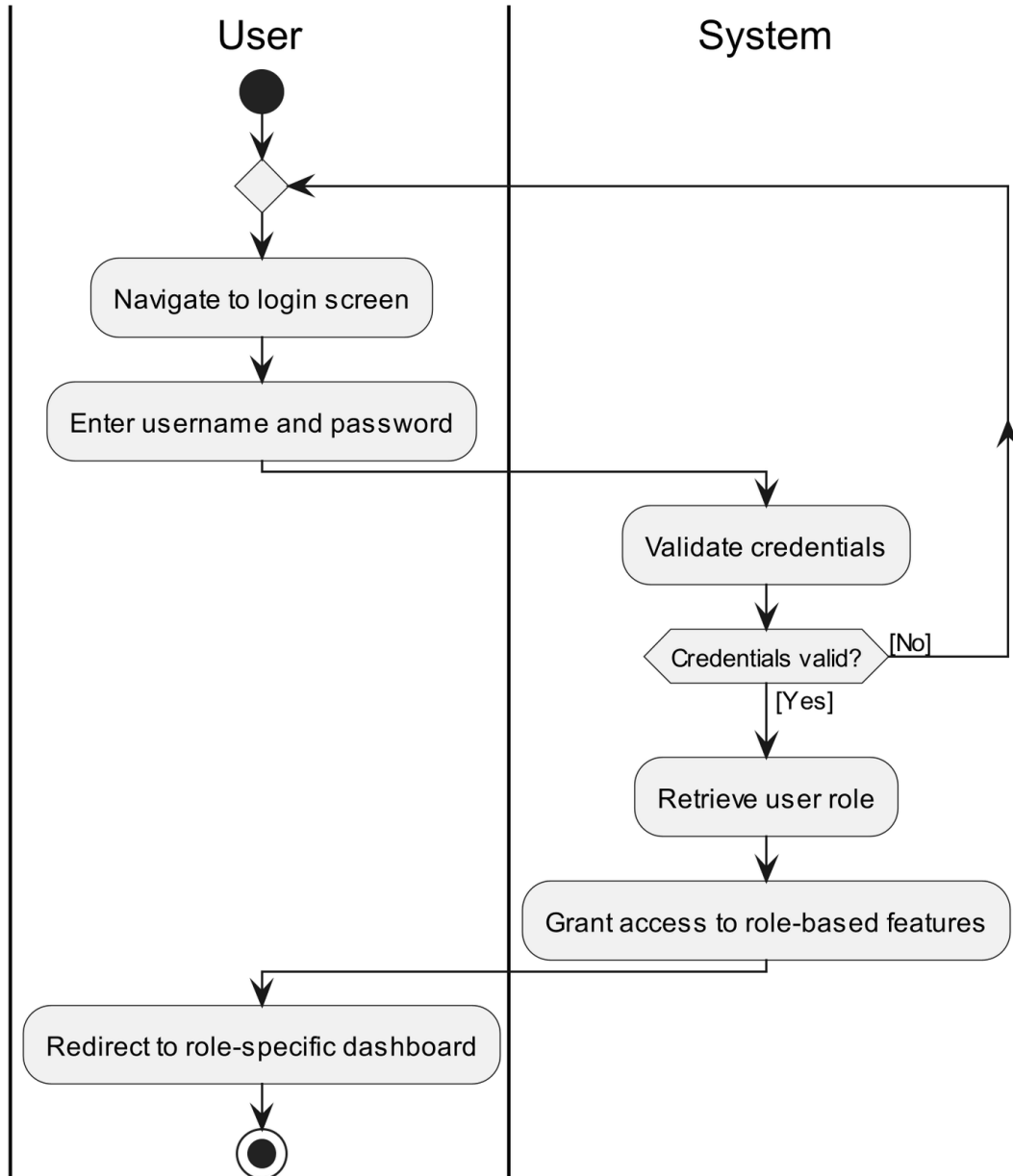
Field	Content
Use Case ID	UC-15
Use Case Name	Monitor Centralized Branch Operations & Analytics
Actor(s)	Restaurant Owner
Preconditions	Owner is logged into the central admin dashboard.
Postconditions – Success	Owner successfully generates and views performance reports.
Postconditions – Failure	Data fails to aggregate; dashboard displays an error.
Main Flow	1. Owner opens the analytics dashboard. 2. System aggregates data from all branches. 3. System displays sales, inventory, and delivery metrics. 4. Owner applies filters (e.g., date range, specific branch). 5. System updates visual charts and data tables.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-15: Analytics dashboard is strictly restricted to the Restaurant Owner role.

2.7 Activity Diagrams

The dynamic aspects of the system were modeled via activity diagrams.

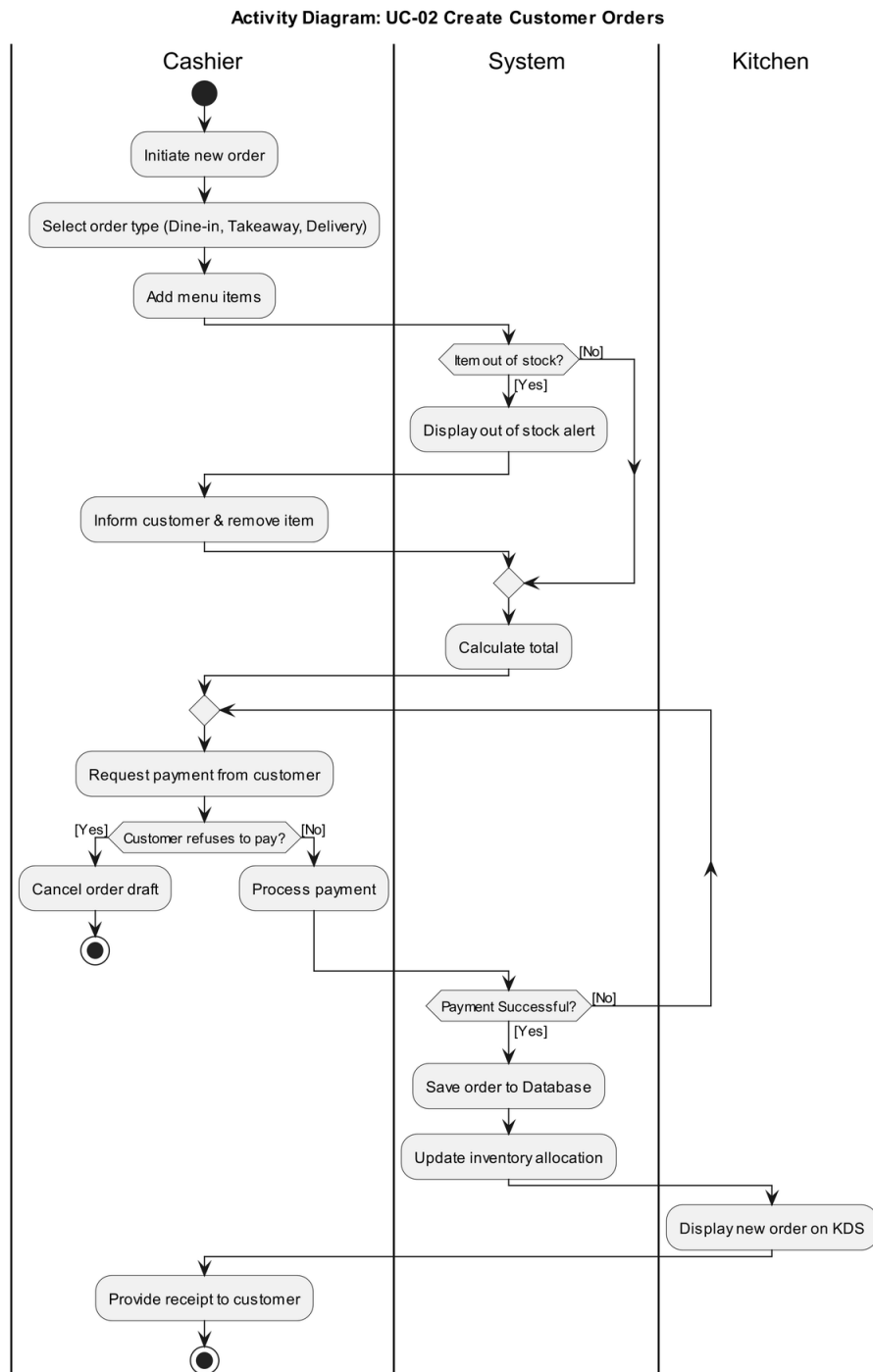
Figure 2.3: Activity Diagram - UC-01

Activity Diagram: UC-01 Log In and Access Role-Based Features



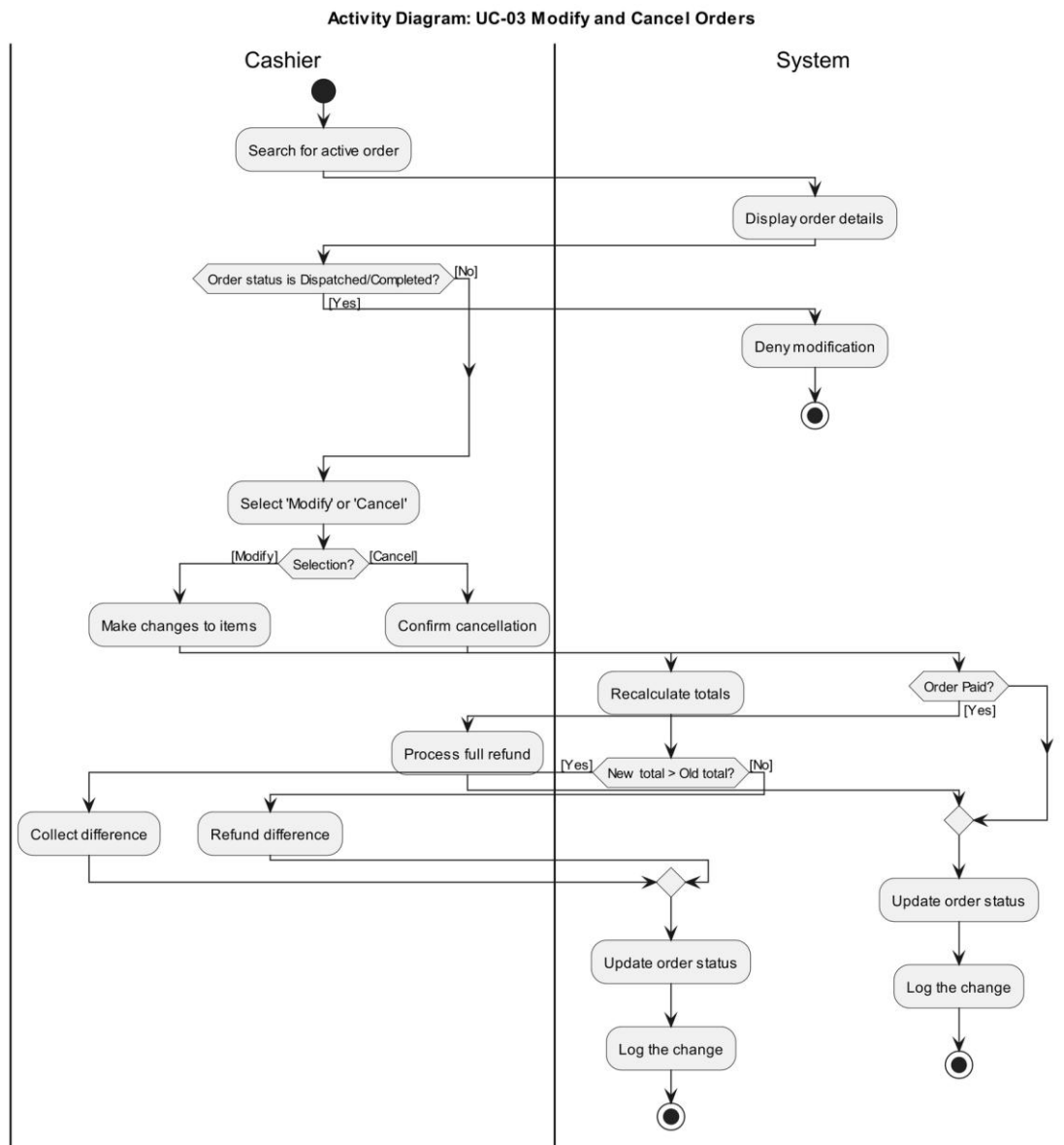
AD-01

Figure 2.4: Activity Diagram - UC-02



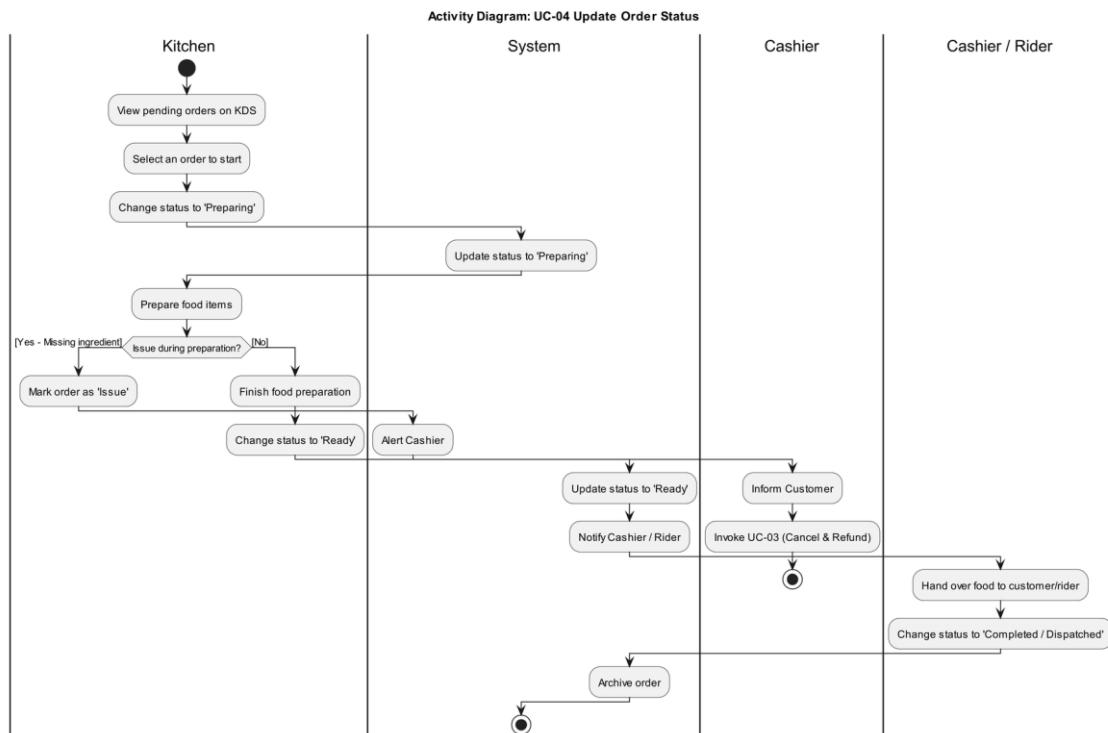
AD-02

Figure 2.5: Activity Diagram - UC-03



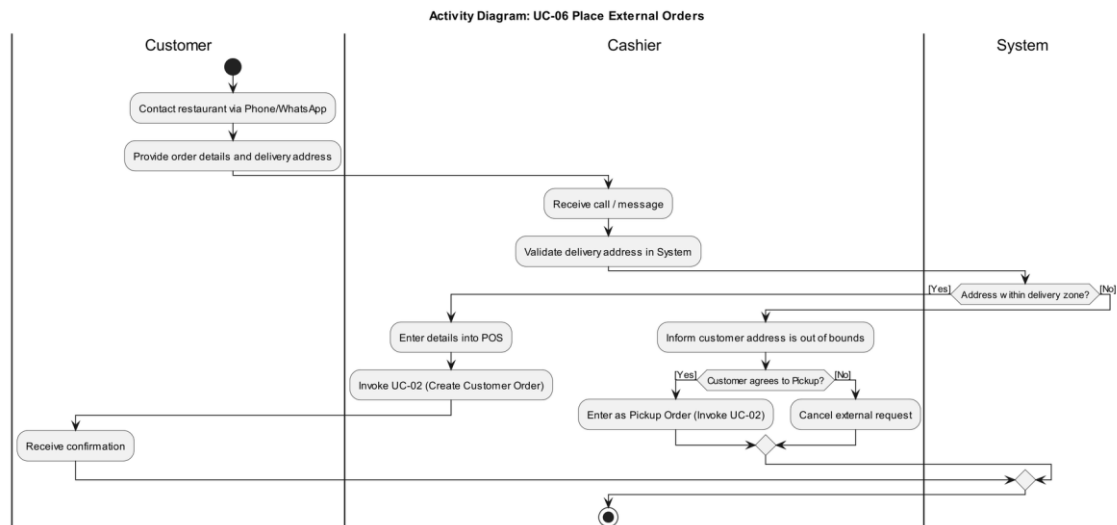
AD-03

Figure 2.6: Activity Diagram - UC-04



AD-04

Figure 2.8: Activity Diagram - UC-06



AD-06

Figure 2.9: Activity Diagram - UC-07

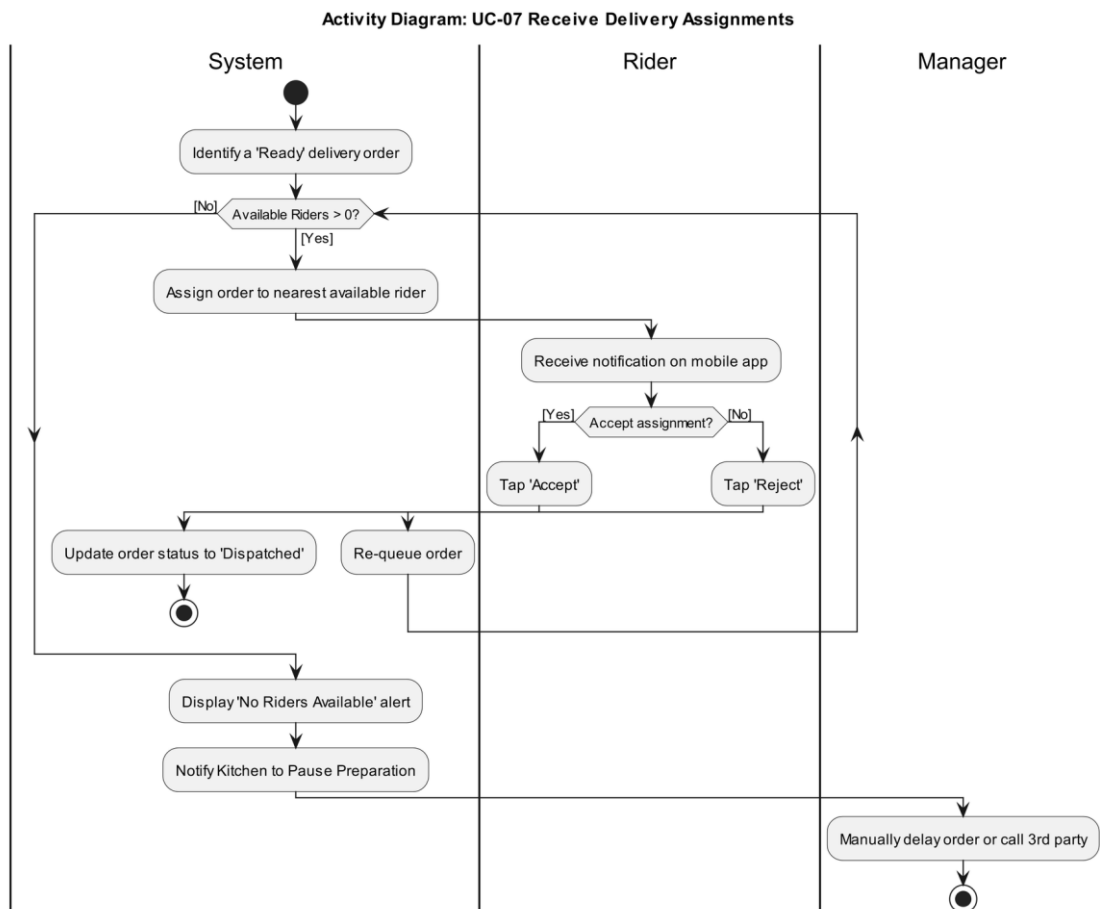


Figure 2.10: Activity Diagram - UC-08

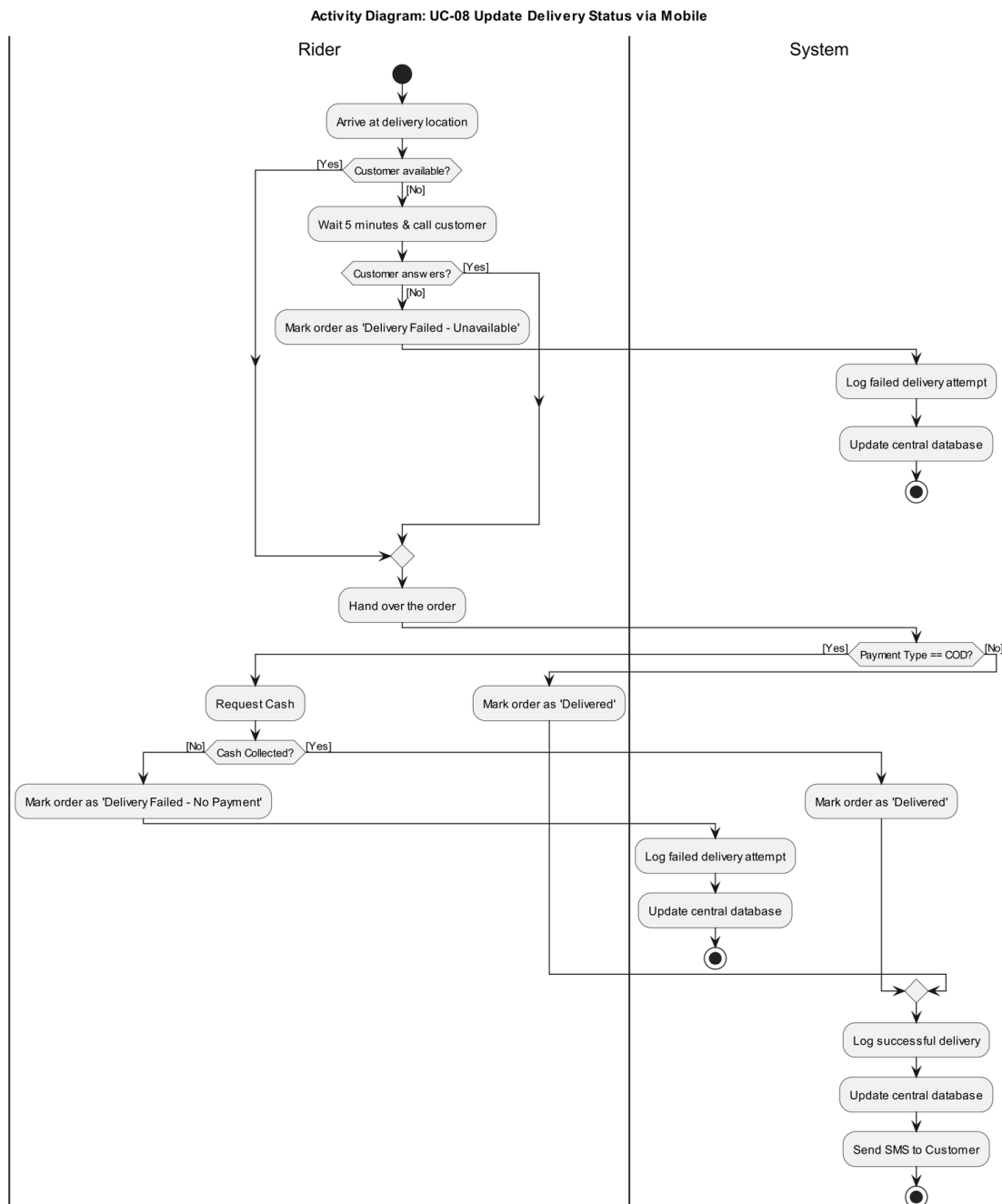
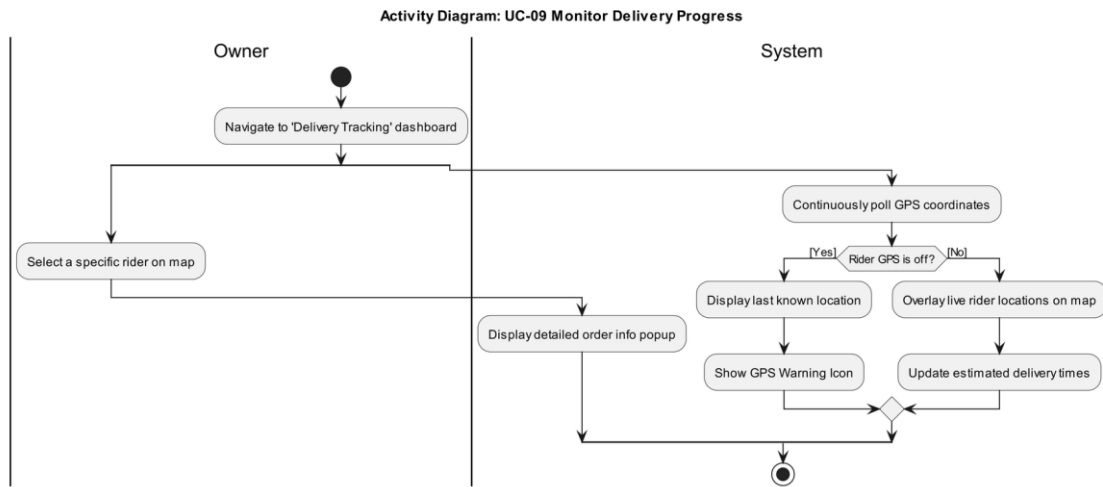
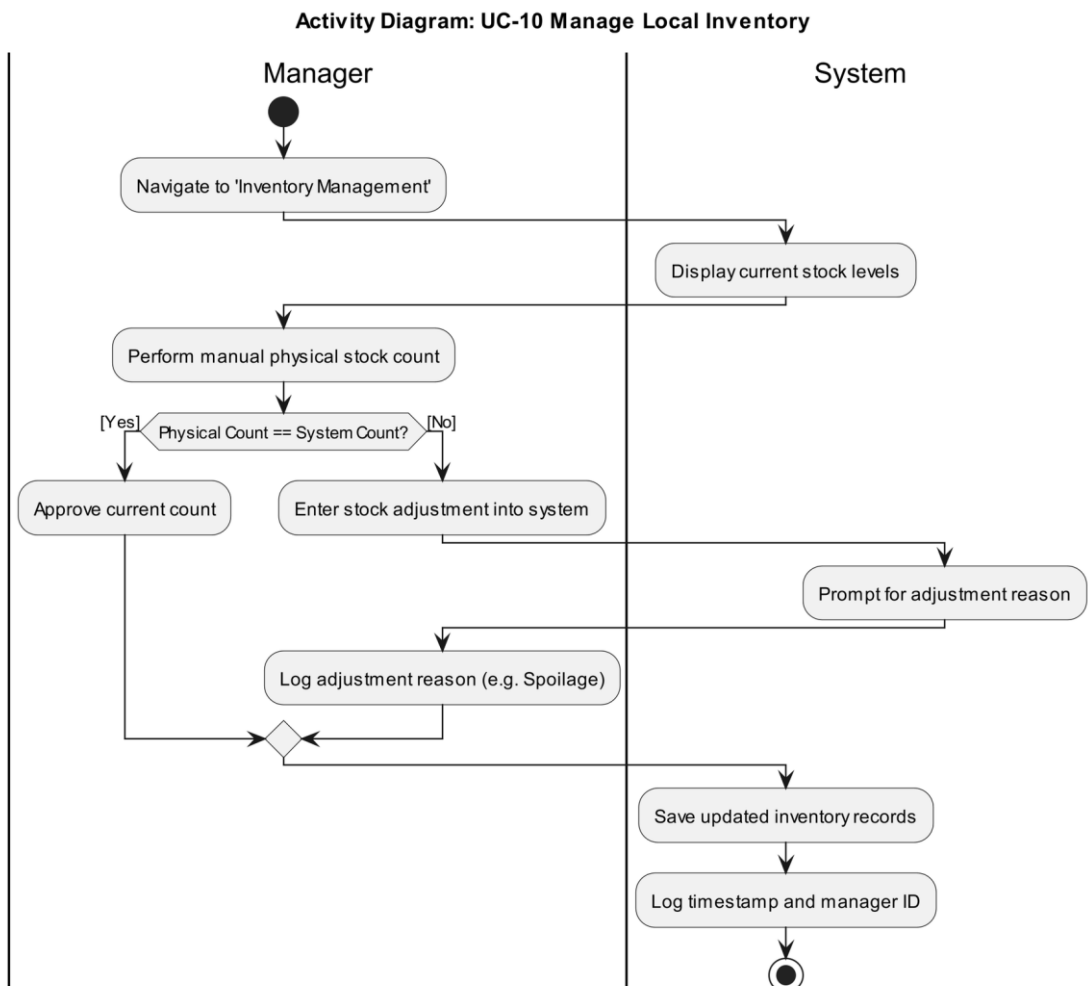


Figure 2.11: Activity Diagram - UC-09



AD-09

Figure 2.12: Activity Diagram - UC-10



2.8 Feasibility Study & Alternatives Considered

Before selecting the proposed custom n-tier architecture, an extensive feasibility study was conducted. Two alternative solutions were evaluated and rejected:

1. Alternative 1: Off-the-Shelf POS (e.g., Square, Toast). This approach was rejected on economic grounds due to prohibitive monthly SaaS fees per branch and transaction commissions which erode margins.
2. Alternative 2: Monolithic Desktop Application. This option was rejected on technical grounds because it could not support the cloud-synced multi-branch dashboard or the mobile Rider App.

CHAPTER 3: SYSTEM DESIGN

3.1 Architectural Decision Records (ADRs)

The system architecture was meticulously designed. The following ADRs document the core technical stack decisions:

ADR-01: Adoption of React.js for Frontend

Context: A highly interactive, fast-loading UI was required for the POS and KDS to minimize order processing latency.

Decision: React.js was selected for all web-based client applications.

Alternatives Considered: Angular (rejected due to steeper learning curve), Vanilla JS (rejected due to lack of state management).

Rationale: Fast rendering is ensured by React's virtual DOM, meeting the <2s KDS sync requirement.

ADR-02: Adoption of Flutter for Rider App

Context: A mix of Android and iOS personal devices is used by delivery riders.

Decision: Flutter was chosen for the Rider Mobile Application.

Alternatives Considered: Native Swift/Kotlin (rejected due to requiring two separate codebases).

Rationale: A unified codebase with near-native performance is provided by Flutter.

ADR-03: Selection of PostgreSQL

Context: Highly relational data (Orders, Invoices, Inventory) is processed by the system.

Decision: PostgreSQL was selected as the primary relational database.

Alternatives Considered: MongoDB (rejected due to lack of strict ACID compliance for financial transactions).

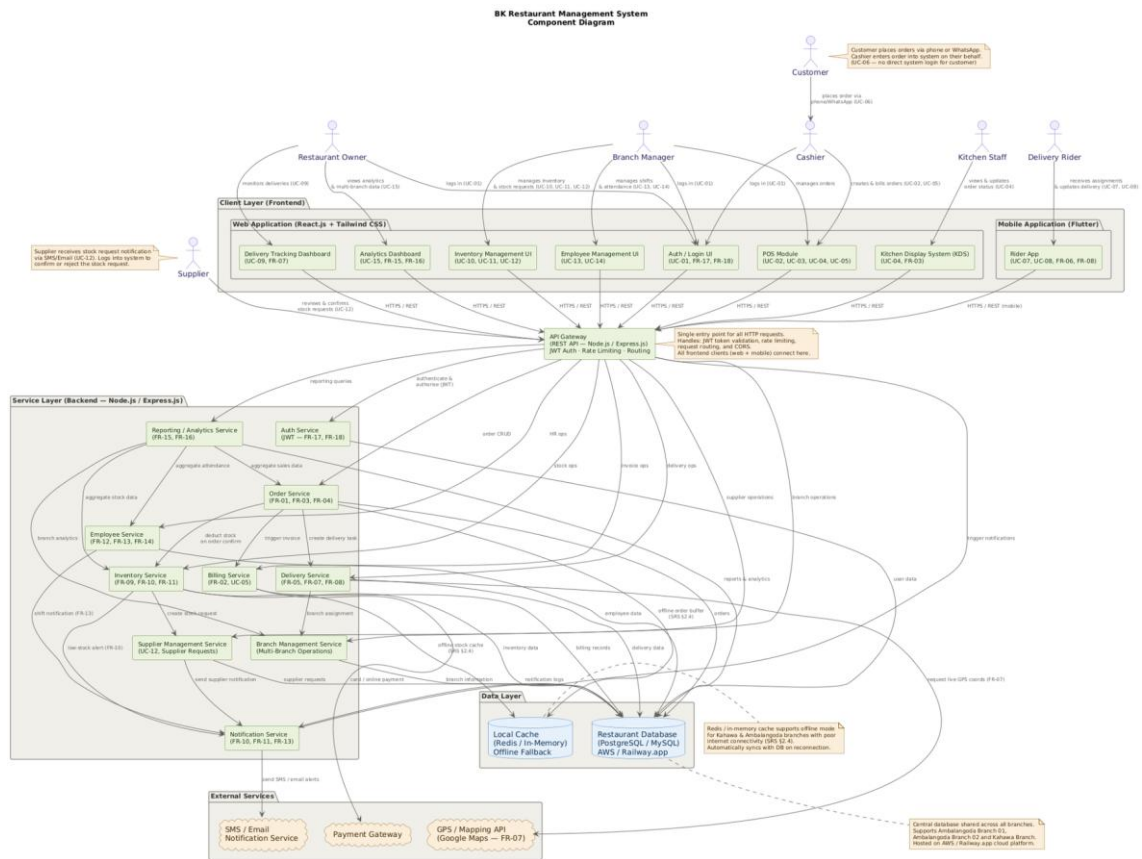
Rationale: ACID compliance is guaranteed by PostgreSQL, which is mandatory for billing integrity.

3.2 System Architecture Diagrams

Figure 3.1: Component Diagram

The comprehensive Component Diagram below illustrates the detailed internal architecture of the system. It maps the exhaustive relationships between all distinct software components, defining the interfaces between the React.js client modules, the individual Node.js backend microservices, and the structured PostgreSQL database tables.

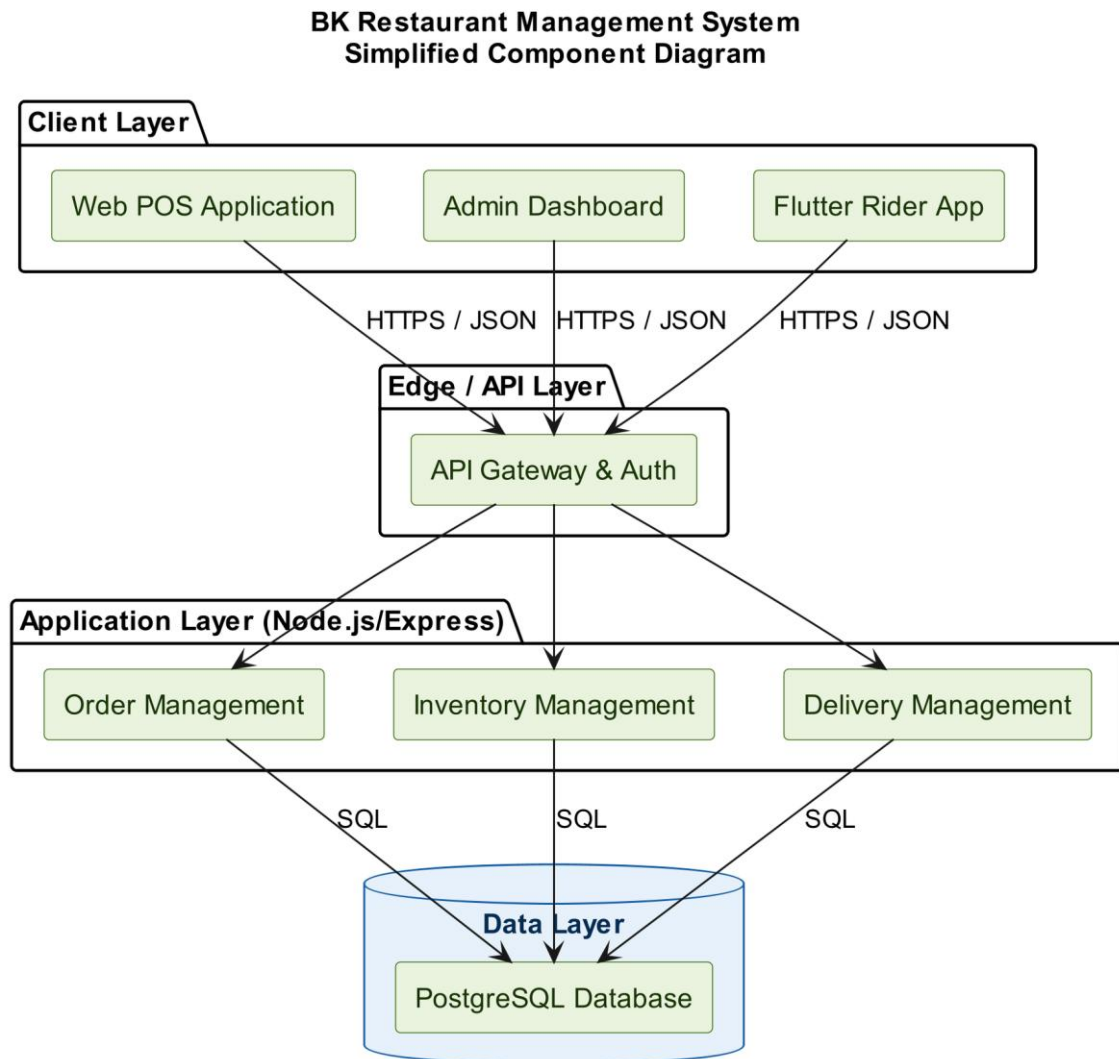
SENG 31242 - BK Restaurant Management System



Component Diagram

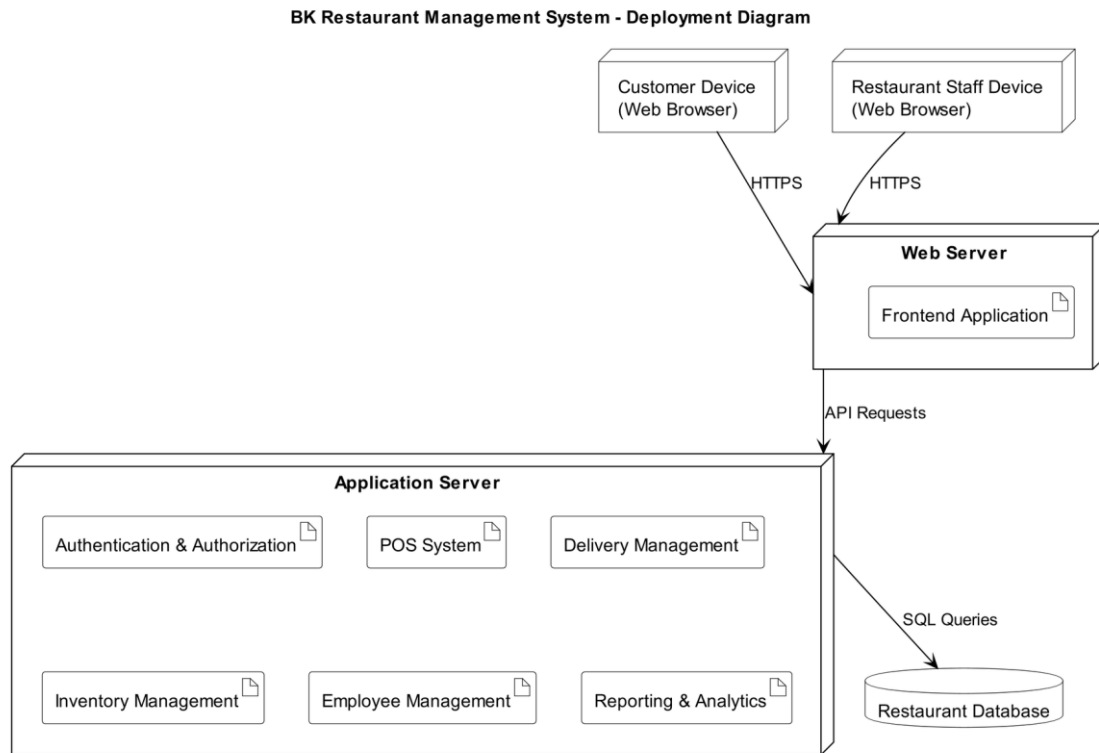
Figure 3.1.1: Simplified Component Diagram

A high-level view highlighting the interactions between the Client Layer, Application Layer, and Data Layer.



Simplified Component Diagram

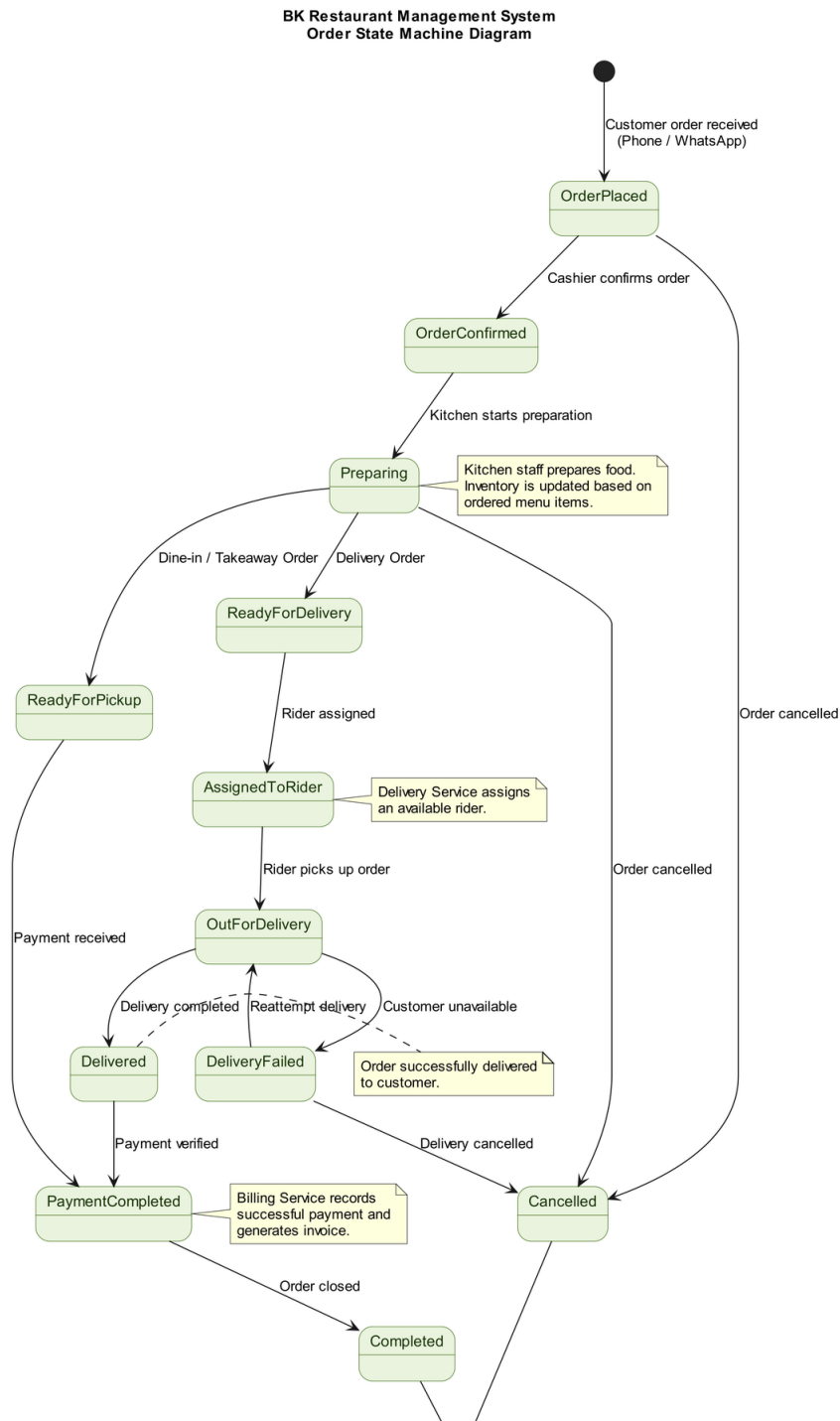
Figure 3.2: Deployment Diagram



Deployment Diagram

The Deployment Diagram above illustrates the physical hardware and software nodes of the BK Restaurant Management System. Client devices access the platform securely via HTTPS through a centralized Web Server, which serves the Frontend Application. The Web Server then routes internal API requests to the Application Server, which hosts the core backend micro-services (POS, Delivery, Inventory). Finally, all persistent transactional data is managed securely by the backend PostgreSQL Database.

Figure 3.3: Class Diagram / ERD State Behaviors



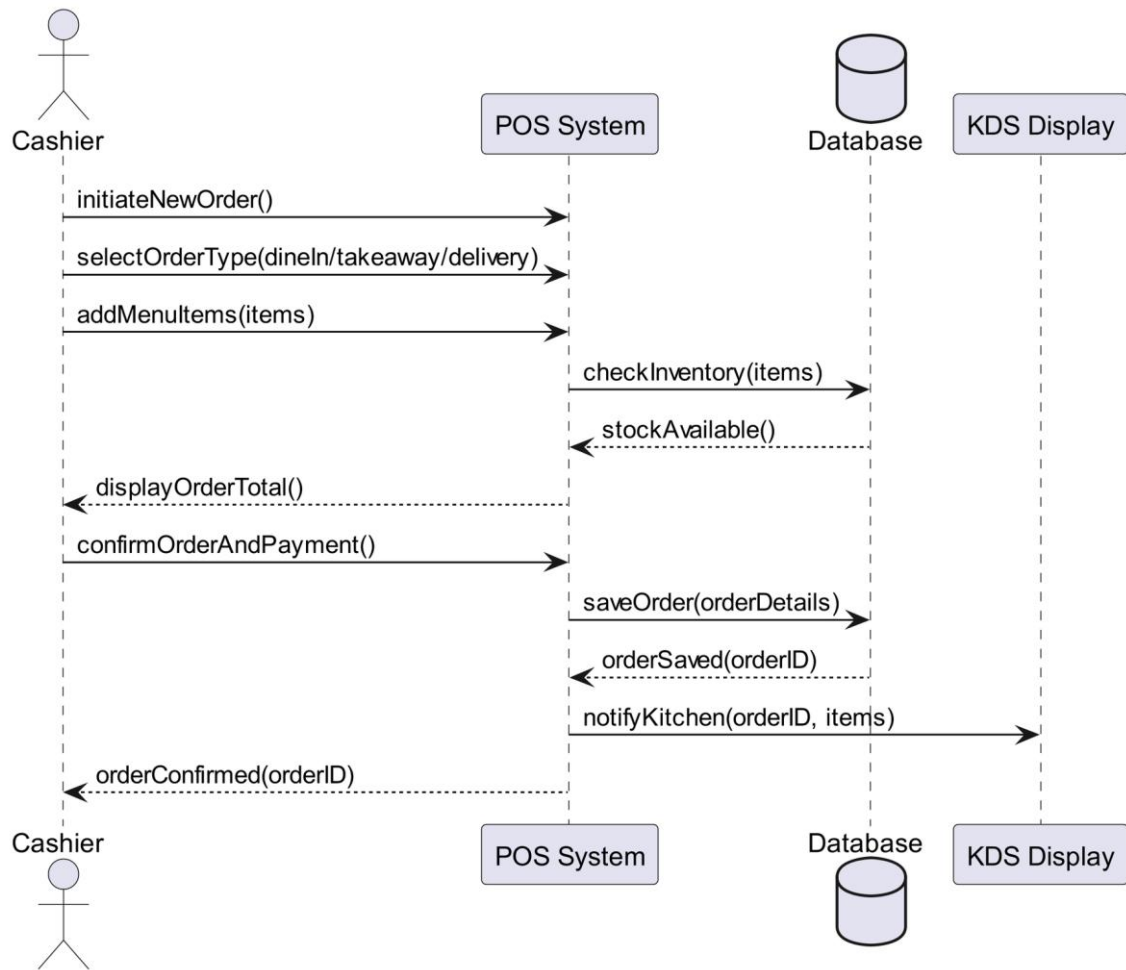
Class Diagram

3.3 Sequence Diagrams

The messaging between system components was documented via Sequence Diagrams.

Figure 3.4: Sequence Diagram - UC-02 (POS Order Entry)

Real-World Scenario: A walk-in customer arrives at the front desk to place a dine-in order. The Cashier uses the Point of Sale (POS) system to log the order type and add menu items. In real-time, the POS verifies ingredient availability against the database. Once payment is confirmed, the order is permanently saved and instantly pushed to the Kitchen Display System (KDS), eliminating the need for physical paper tickets.



SD-02

Figure 3.5: Sequence Diagram - UC-03 (Modify/Cancel Order)

Real-World Scenario: A customer changes their mind immediately after ordering and wants to add a beverage. The Cashier searches for the active order ID. The system fetches the current state, and the Cashier modifies the items. The backend dynamically recalculates the total, updates the database, and synchronizes the modification so the kitchen is immediately aware of the change.

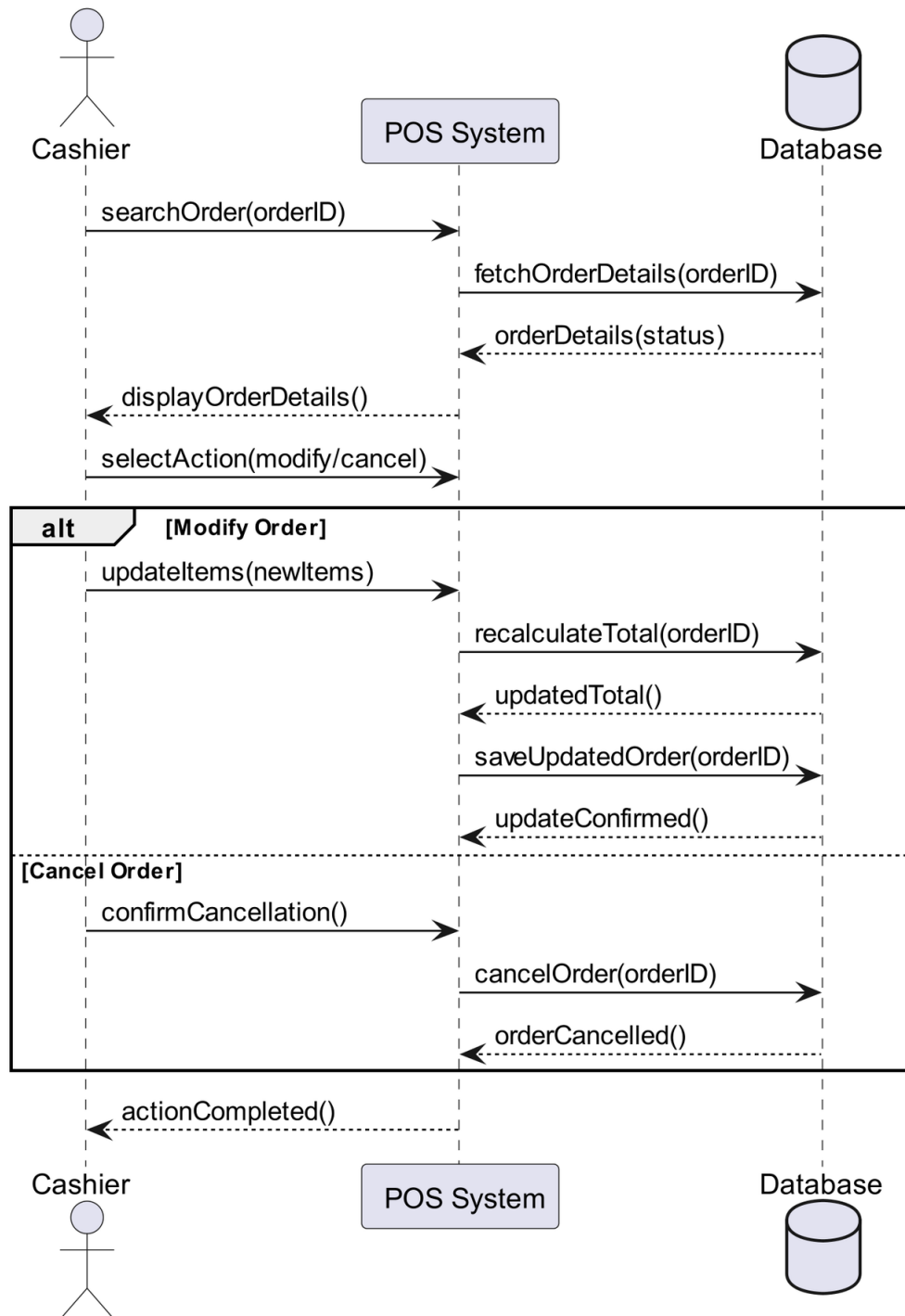


Figure 3.6: Sequence Diagram - UC-04 (Kitchen Order Processing)

Real-World Scenario: In the bustling kitchen environment, the Head Chef monitors the KDS display which continuously polls for active orders. When an order is completed, the chef taps the screen to change its status to "Ready". This triggers a backend update that marks the order as ready for pickup or dispatch, ensuring seamless synchronization between the kitchen and the front-of-house staff.

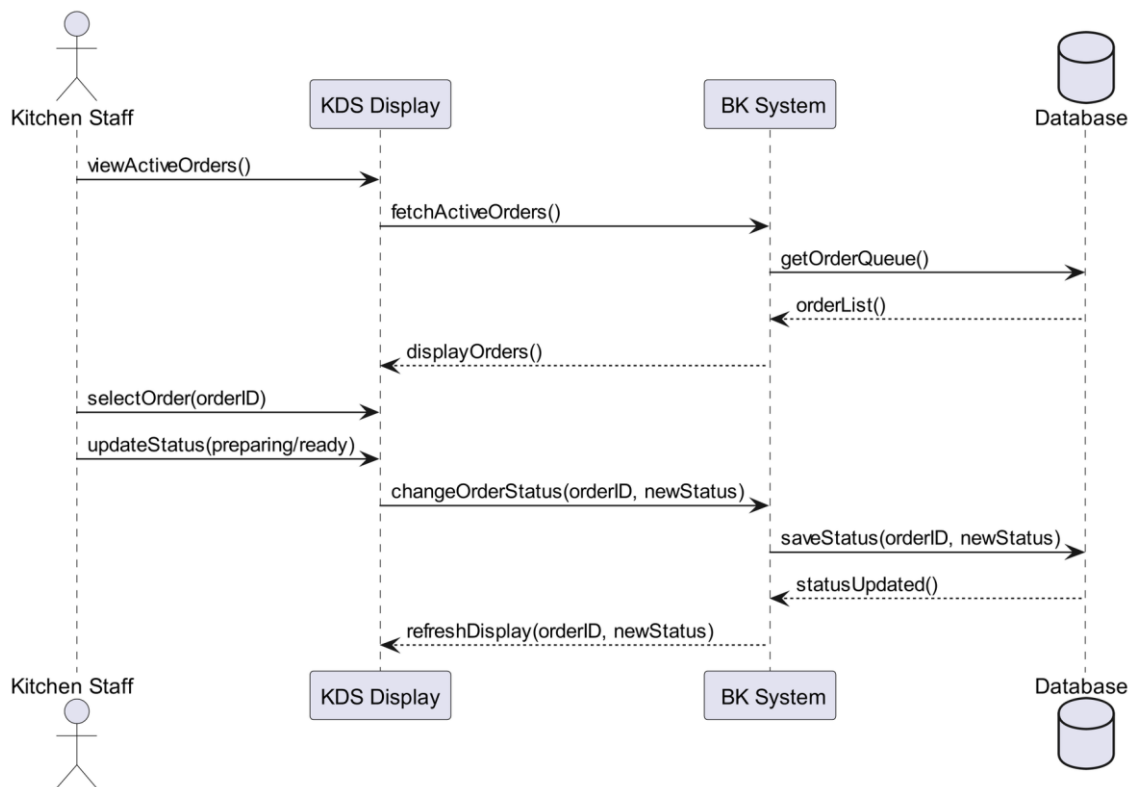
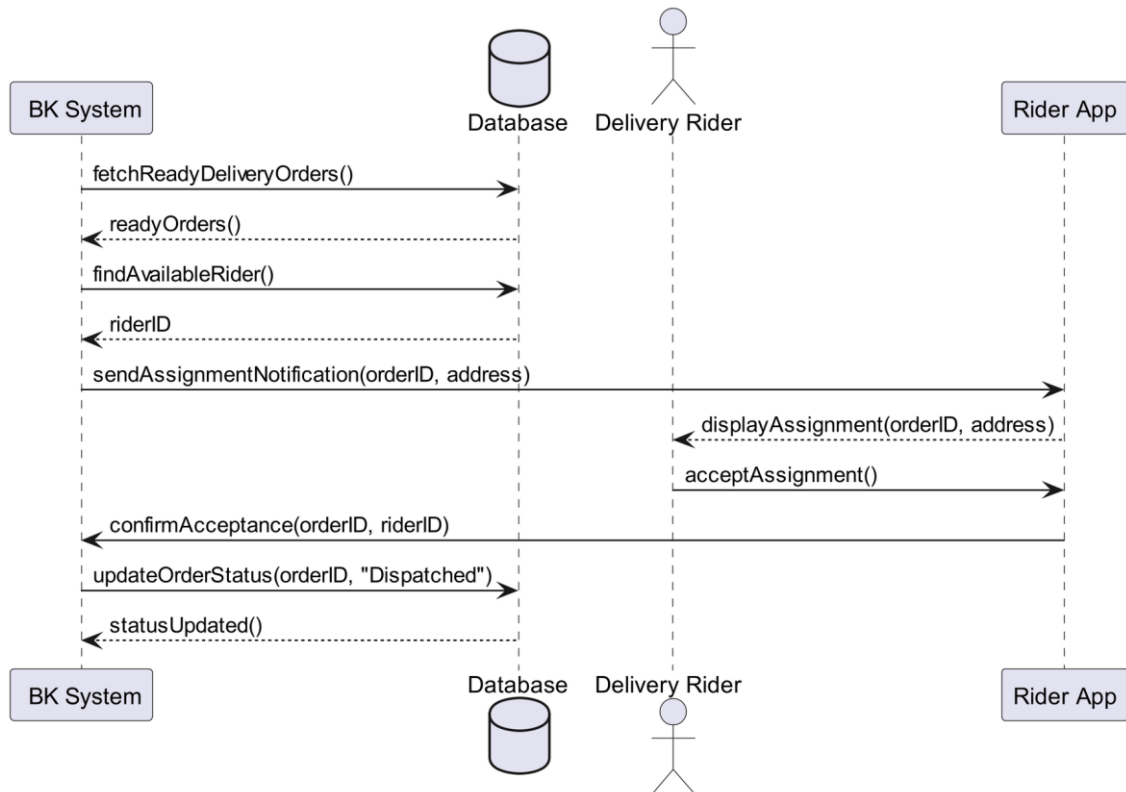


Figure 3.7: Sequence Diagram - UC-07 (Rider Assignment)

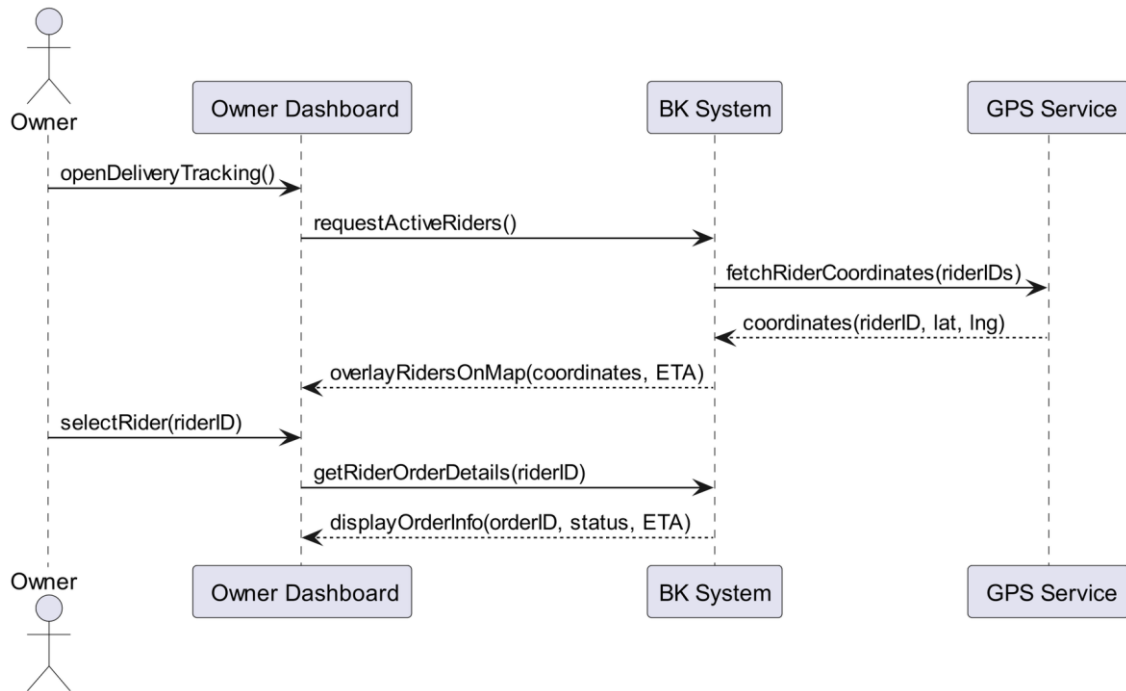
Real-World Scenario: An external takeaway order is marked as "Ready" by the kitchen. The central system automatically scans the database for available delivery riders. A push notification is sent to the designated Rider's mobile app. Once the Rider taps "Accept" on their device, the system updates the backend status to "Dispatched", handing off the chain of custody to the delivery fleet.



SD-07

Figure 3.8: Sequence Diagram - UC-09 (Live Delivery Tracking)

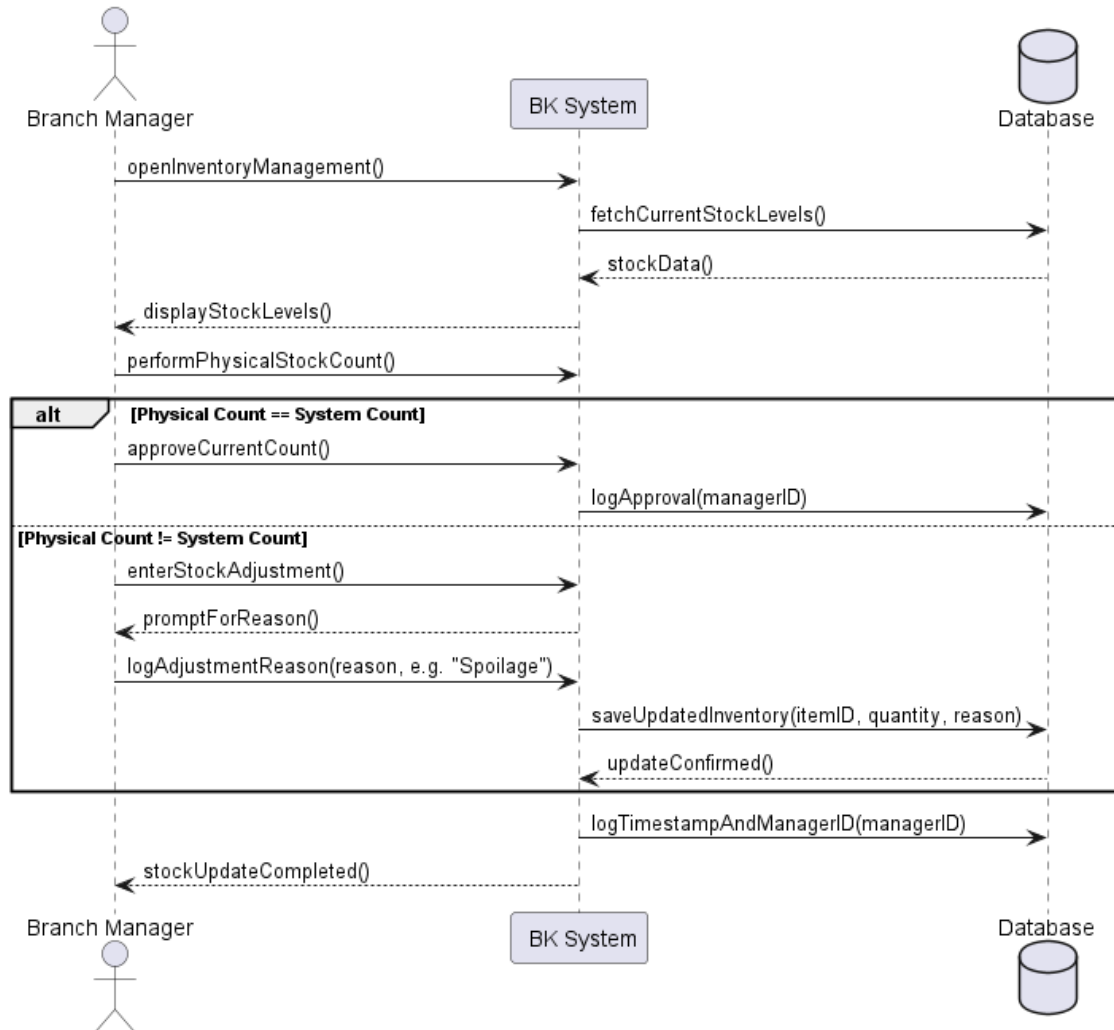
Real-World Scenario: The Restaurant Owner wants to check the efficiency of the delivery fleet during peak hours. By opening the Owner Dashboard, the system requests live telemetry data. The backend fetches continuous GPS coordinates from the external GPS service and streams them back to the frontend, plotting the active riders on a live map interface with calculated ETAs.



SD-09

Figure 3.9: Sequence Diagram - UC-10 (Inventory Adjustment)

Real-World Scenario: During an end-of-day audit, a Branch Manager discovers that a bag of flour was damaged. The Manager navigates to the Inventory module, fetches current stock levels, and enters a manual stock adjustment along with the reason ("Spoilage"). The system immutably logs the transaction against the Manager's ID for strict auditing purposes and updates the master inventory.



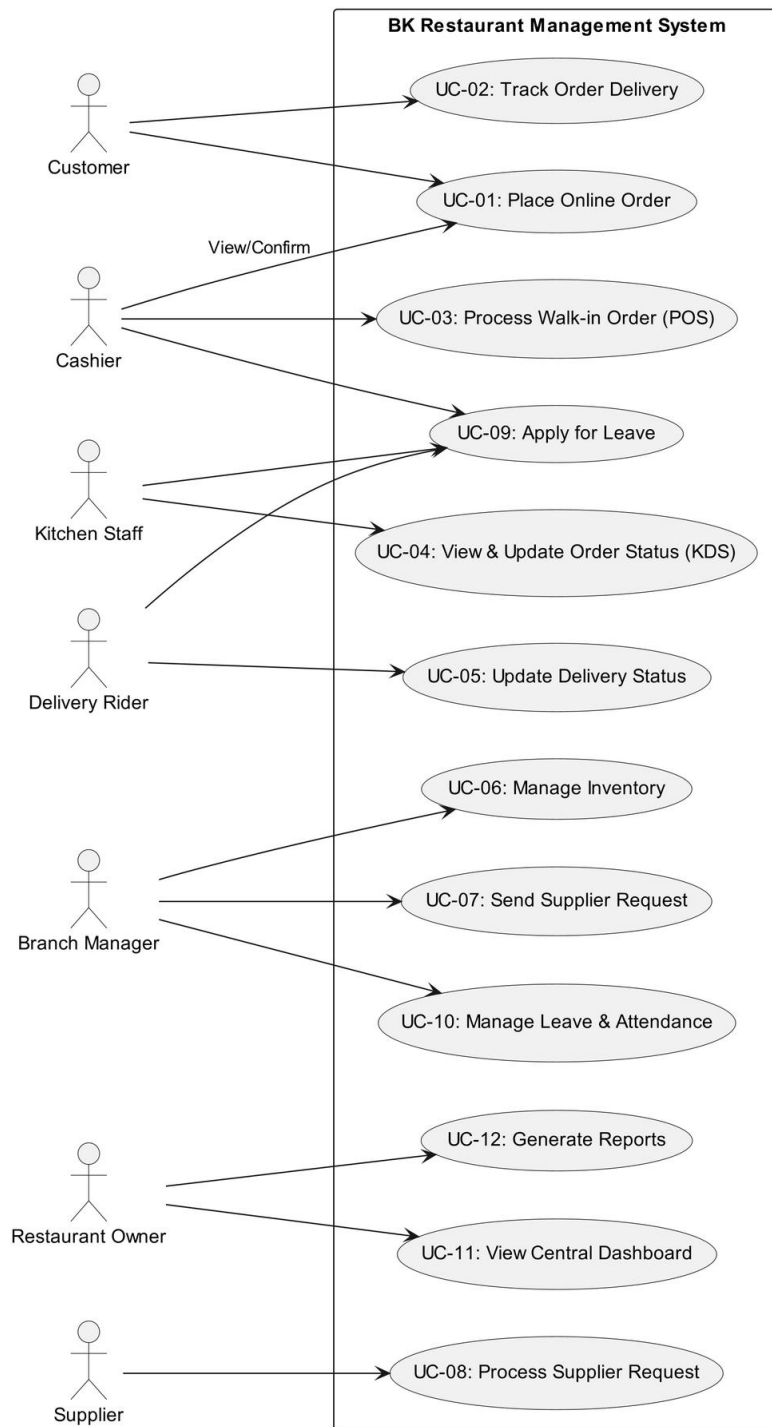
SD-10

3.4 Use Case Diagrams

Figure 3.10: Use Case Diagram (Role-Based Access)

Description: This Use Case diagram maps the comprehensive role-based access controls across the system. It illustrates exactly which features (Use Cases) are available to each

specific user role (Owner, Manager, Cashier, Kitchen, Rider), serving as the foundational navigational map for the UI flows.



3.5 User Interface Design & Flow Mockups

The user experience was mapped through comprehensive wireframes and flows.

3.5.1 Navigational User Flows

Figure 3.11: Customer Order Creation Flow

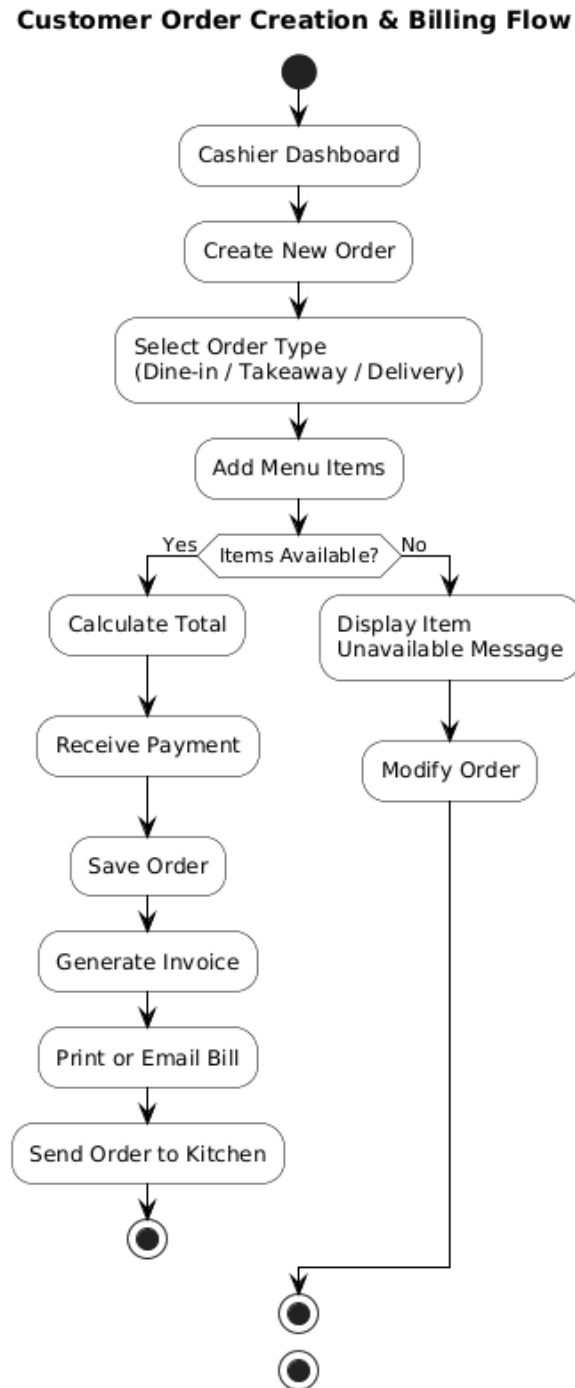


Figure 3.12: Kitchen Order Processing Flow

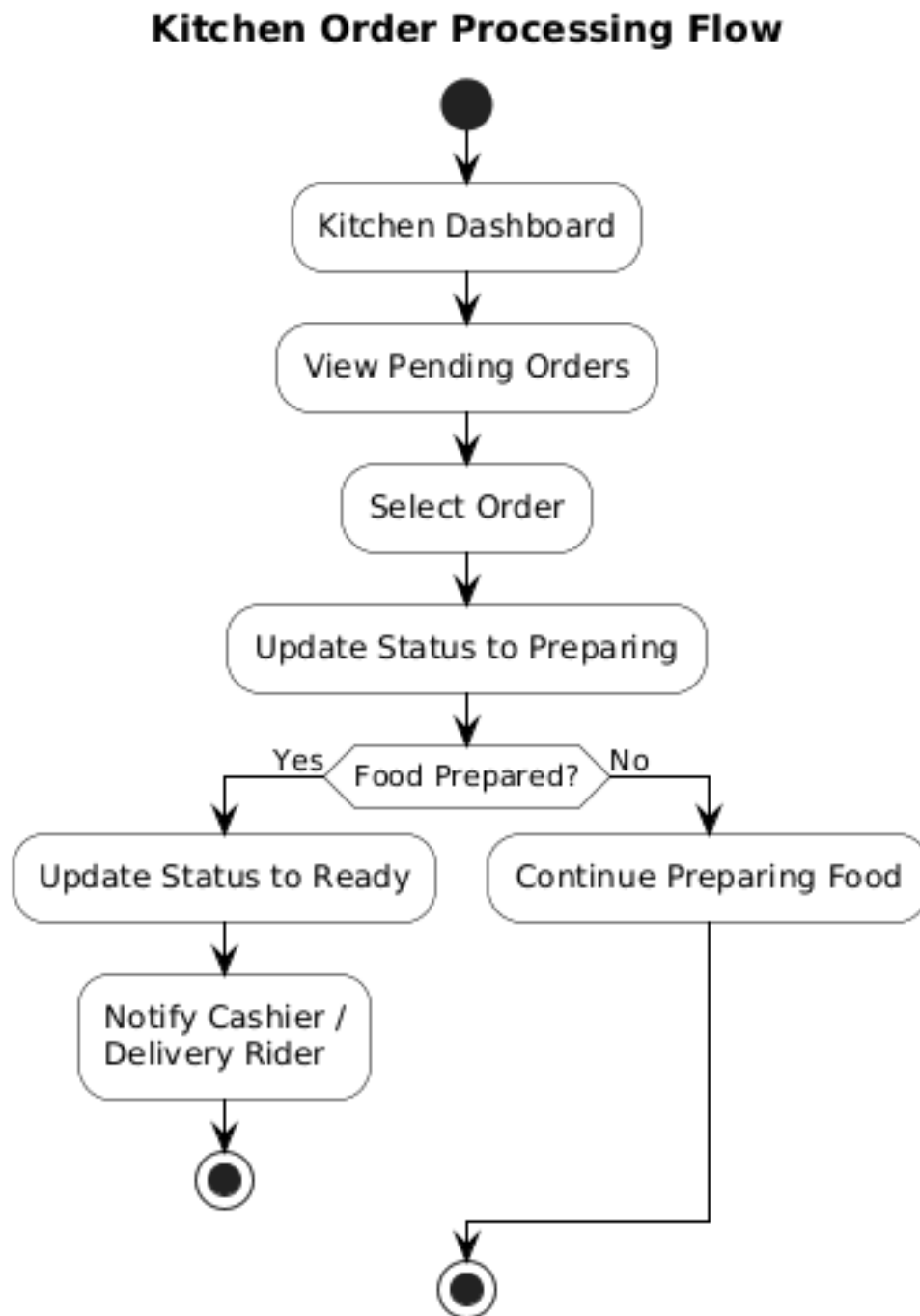


Figure 3.13: Delivery Rider Flow

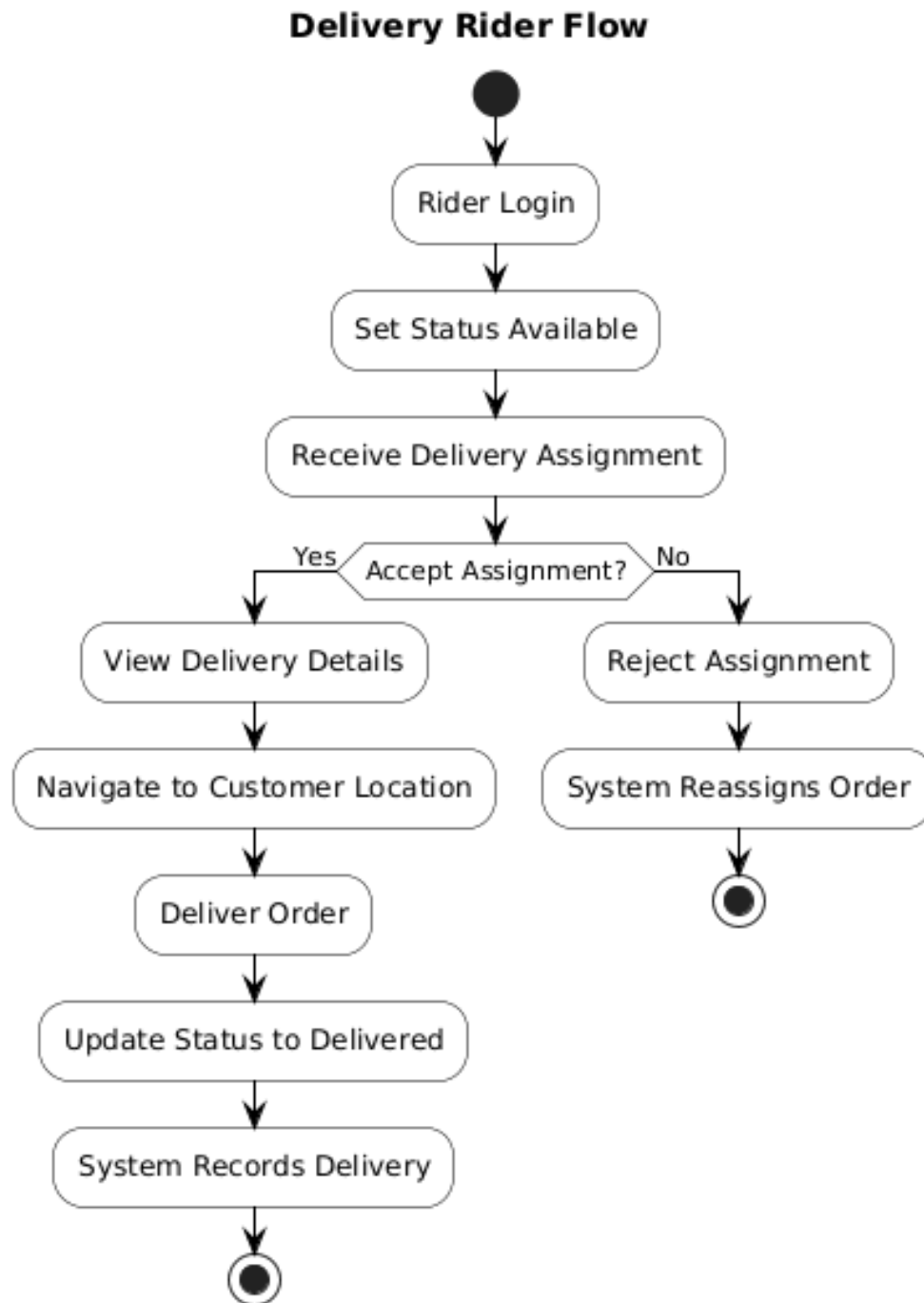


Figure 3.14: Inventory Management Flow

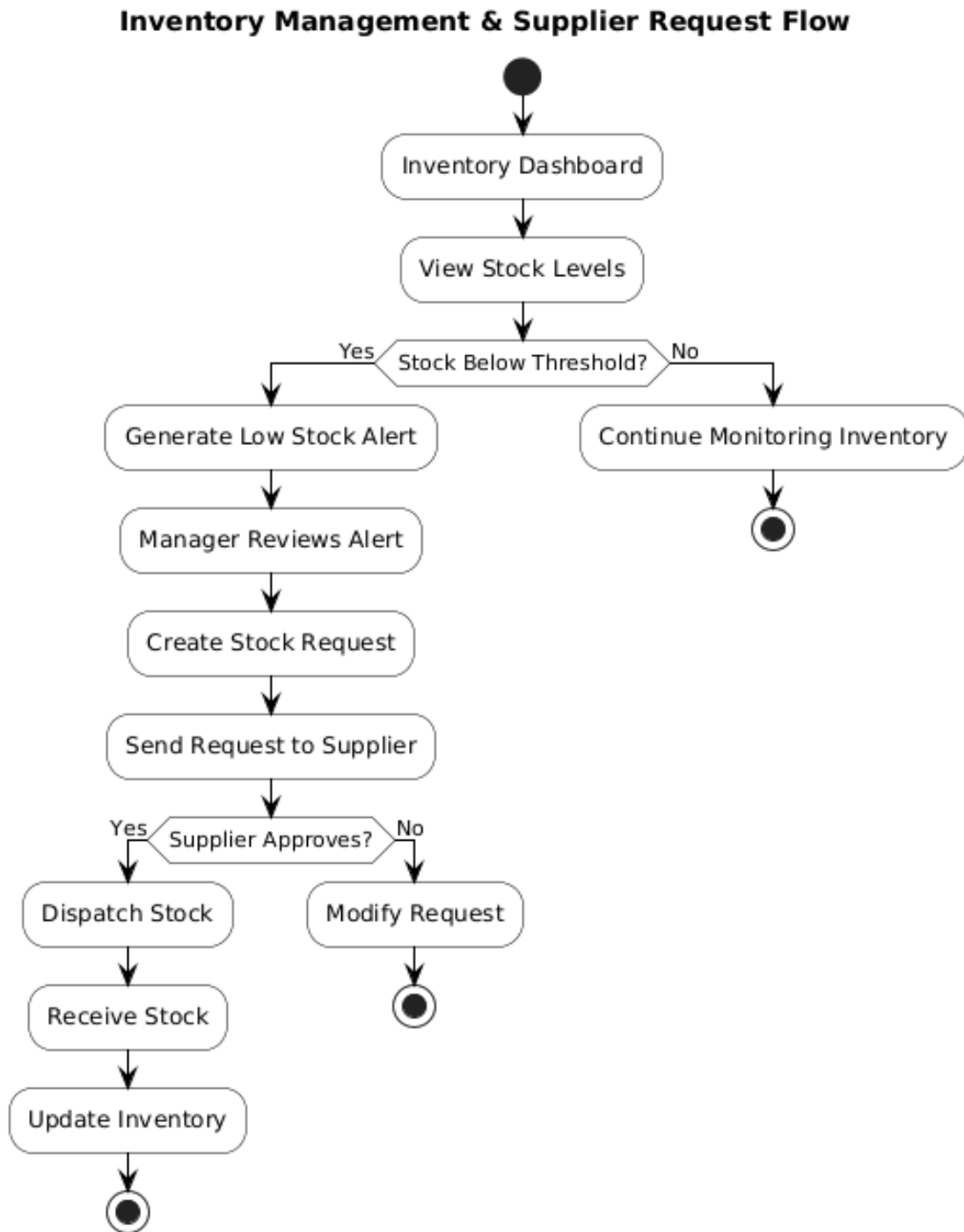
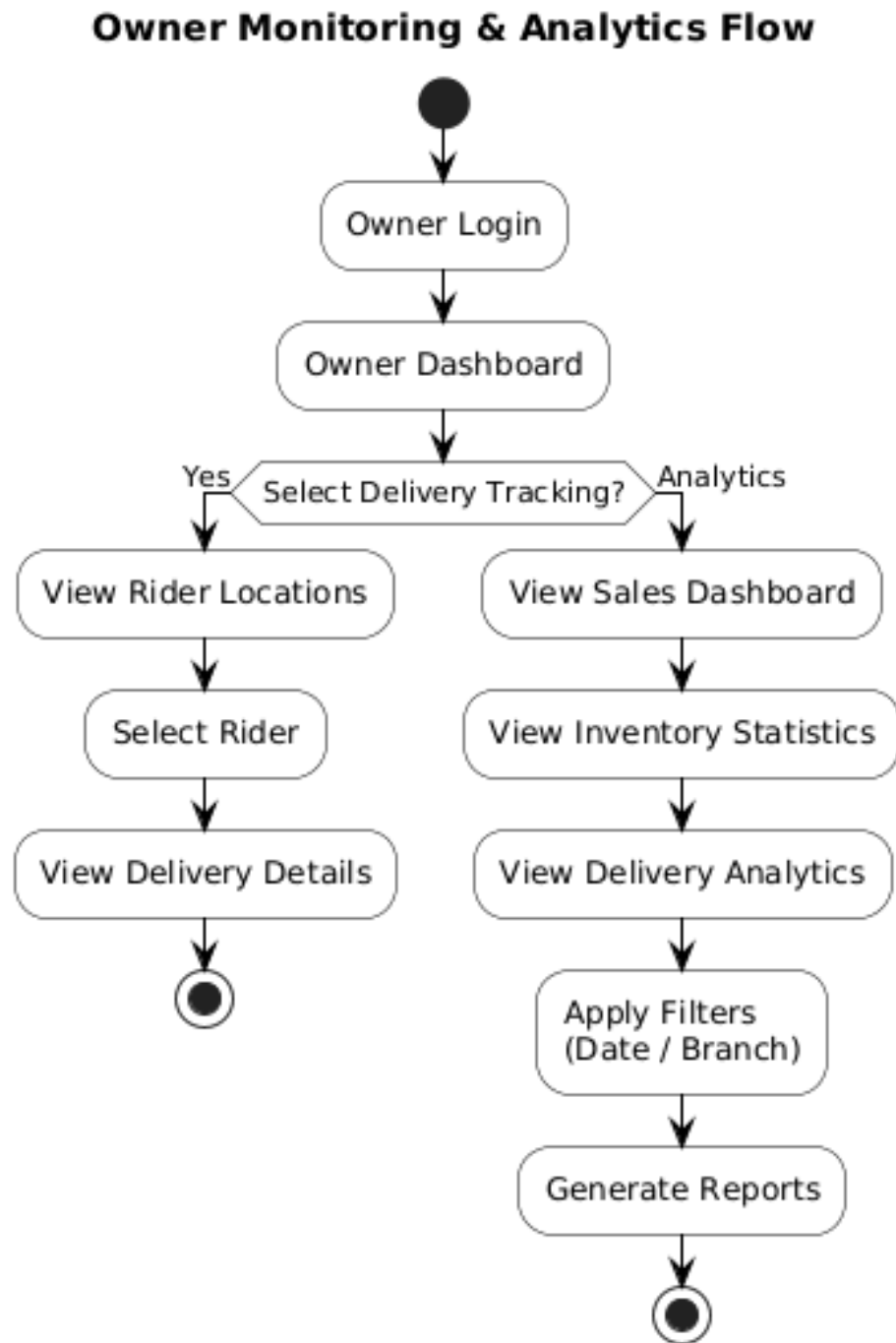


Figure 3.15: Owner Monitoring Flow



3.5.2 UI Screen Mockups

Figure 3.16: UI - Staff Registration

Staff Registration

BK Management
Sri Lankan Flavors

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

Settings

Support

Staff Management

Search staff...

Registration Progress

1 Personal Details
IDENTITY INFORMATION

2 Work Placement
ROLE & BRANCH

3 Security Access
LOGIN CREDENTIALS

Your data is encrypted and stored in compliance with restaurant security protocols.

New Staff Profile
Step 1: Define the basic identity

First Name
e.g. Aruni

Last Name
e.g. Perera

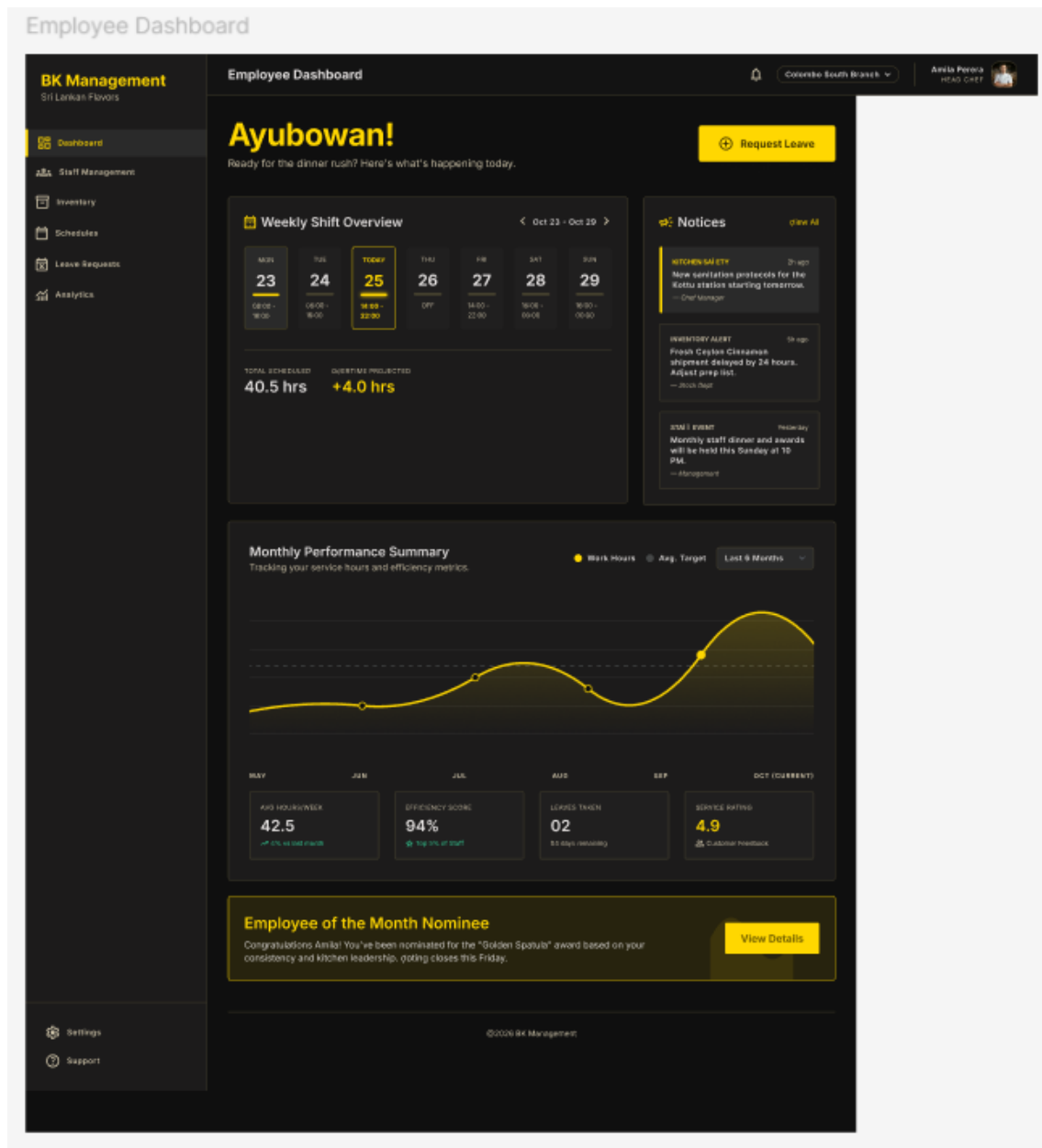
Phone Number
+94 77 123 4567

NEXT STEP ->

SERVER STATUS: OPERATIONAL

© 2026 BK RESTAURANT GROUP. SECURED BY AES-256 ENCRYPTION.

Figure 3.17: UI - Employee Dashboard



UI - Employee Schedule

BK Management

Sri Lankan Flavors

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

New Entry

Settings

Support

Schedules

Columbo Port Branch

Search staff or shift...

Logout

Shift Timetable

August 2024 • Week 34

Weekly Monthly Team View

Today

ACTIVE NOW

Morning Shift

06:00 AM — 02:00 PM

On duty with you

+2.5h this week

42 Hours

Scheduled Total (Week 34)

Calculated base

LKR 28,400

Estimated Weekly Earnings

MON 19	TUE 20	WED 21 (TODAY)	THU 22	FRI 23	SAT 24	SUN 25
MORNING 06:00 - 14:00 Male Kitchen	EVENING 14:00 - 22:00 Floor Staff	ACTIVE NOW 06:00 - 14:00 Pastry Section	NIGHT 22:00 - 06:00 Inventory Check	MORNING 06:00 - 14:00 Male Kitchen	REQUIRED OFF	STORE CLOSED

Team on Duty - Today

Arjun Mendis

Sous Chef

Nimal Perera

Head Server

Borana Kuruppu

Evening Dish

Prithi Silva

Night Dish

Using the Hustle

"Success is served hot. Manage your time like a master chef manages his kitchen - with precision and passion."

UI - Leave Requests

Leave Requests

BK Management
Sri Lankan Flavours

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

Settings

Support

Leave Management

Search requests...

Annual Leave Balance

14 / 21 days

Sick Leave Used

03 / 07 days

Casual Leave

02 / 05 days

Next Holiday

Poya Day - Aug 19

New Request

Leave Type

Annual Leave

From Date

mm/dd/yyyy

To Date

mm/dd/yyyy

Reason

Briefly explain the reason for your leave...

Submit Request

Status Tracking

View All

TYPE	DATES	REASON	STATUS
Annual	Oct 12 - Oct 15 4 Days	Family trip to Kar	APPROVED
Sick	Sep 04 - Sep 05 2 Days	Flu symptoms	APPROVED
Casual	Aug 28 - Aug 28 1 Day	Urgent bank wor	PENDING
Annual	Aug 10 - Aug 12 3 Days	Wedding celebra	REJECTED
Casual	Jul 20 - Jul 20 1 Day	Personal matters	APPROVED

Leave Policy

Please ensure all leave requests are submitted at least 48 hours in advance for planning purposes. Emergency sick leave should be notified by phone before the shift start.

Download Full Policy PDF →

Upcoming Public Holidays

Nikini Full Moon Poya Day

Aug 19

Milad-un-Nabi

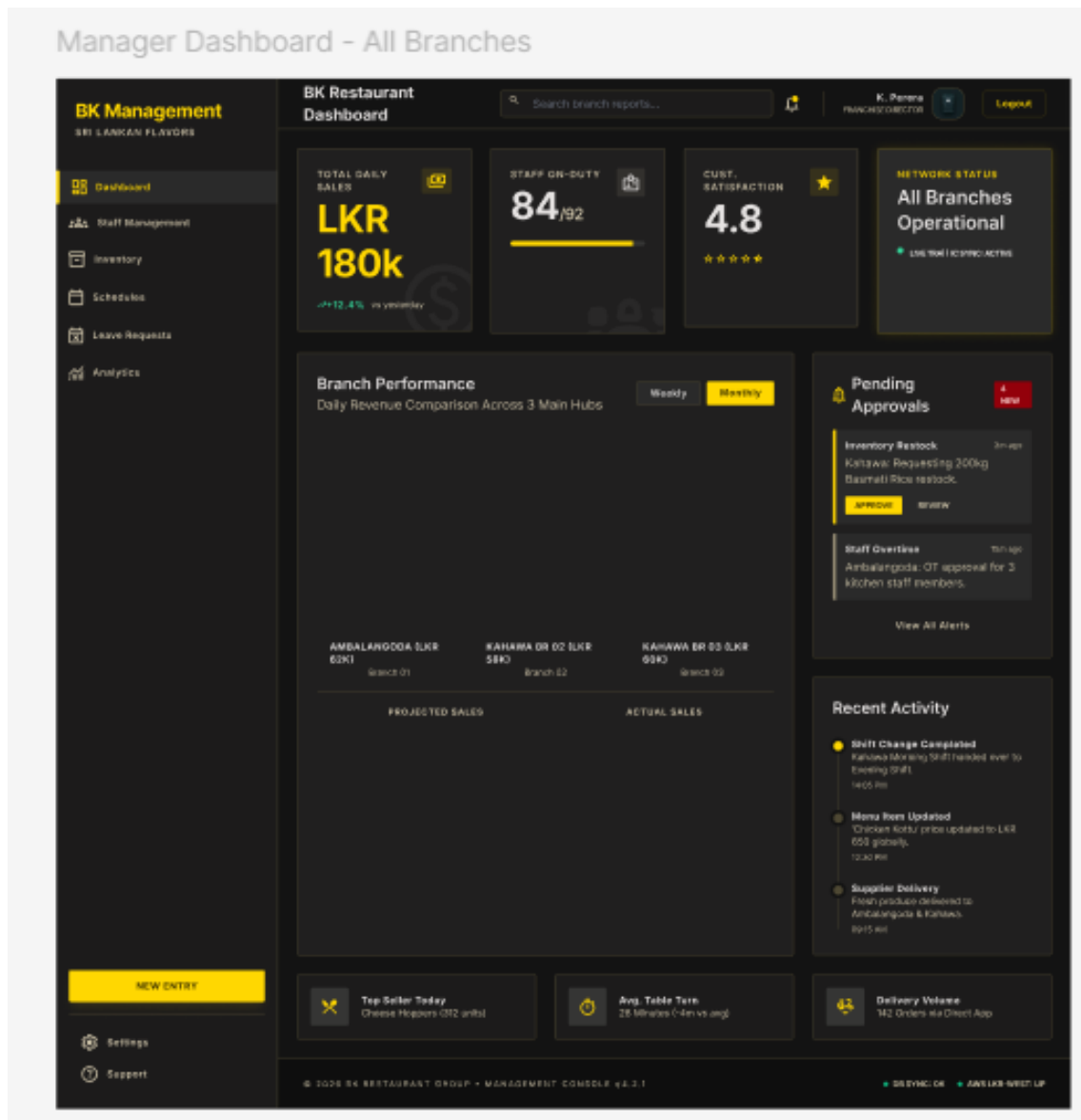
Sep 16

Binara Full Moon Poya Day

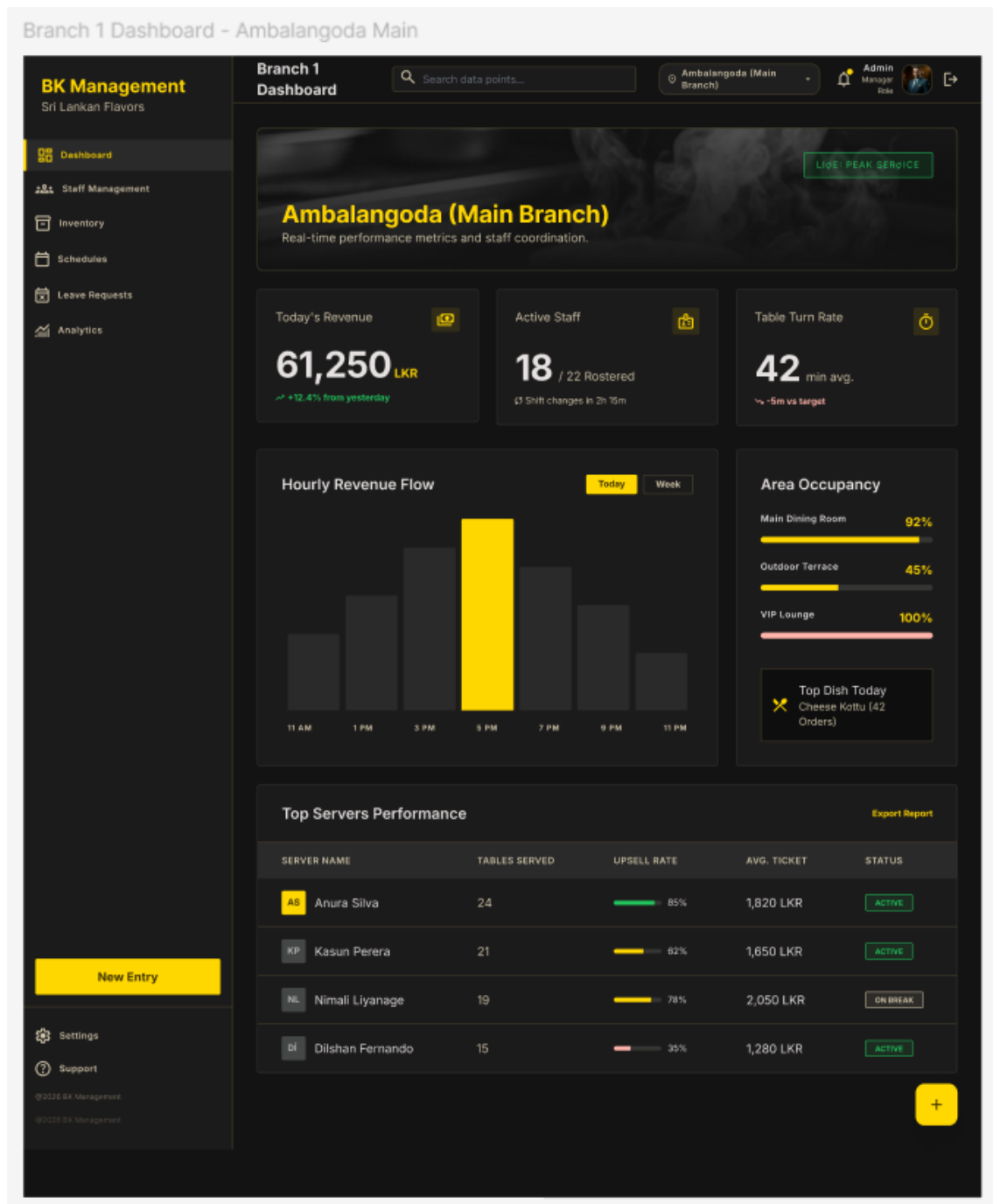
Sep 17

©2026 BK MANAGEMENT

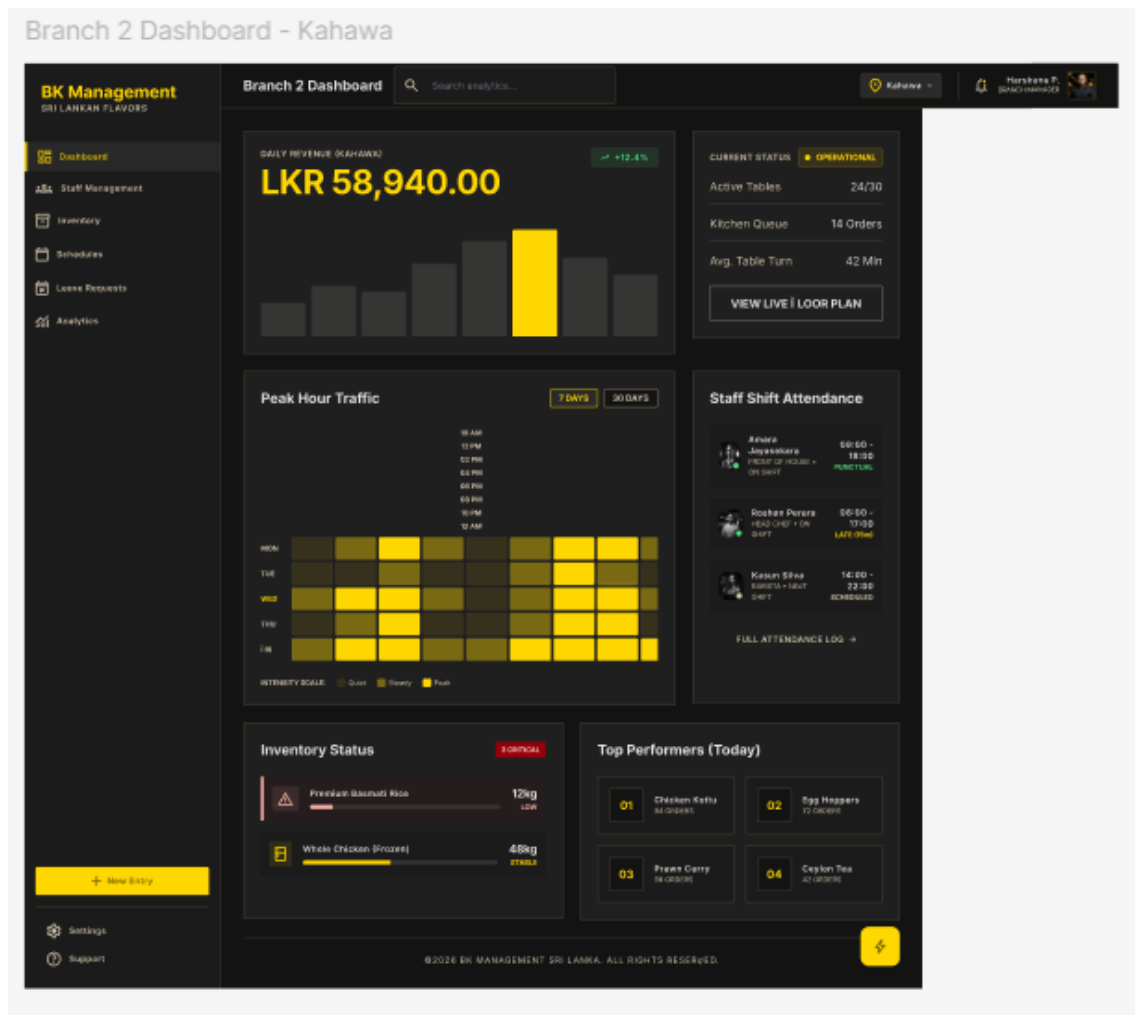
Figure 3.18: UI - Manager Dashboard (Global)



UI - Branch 1 Dashboard



UI - Branch 2 Dashboard



UI - Branch 3 Dashboard

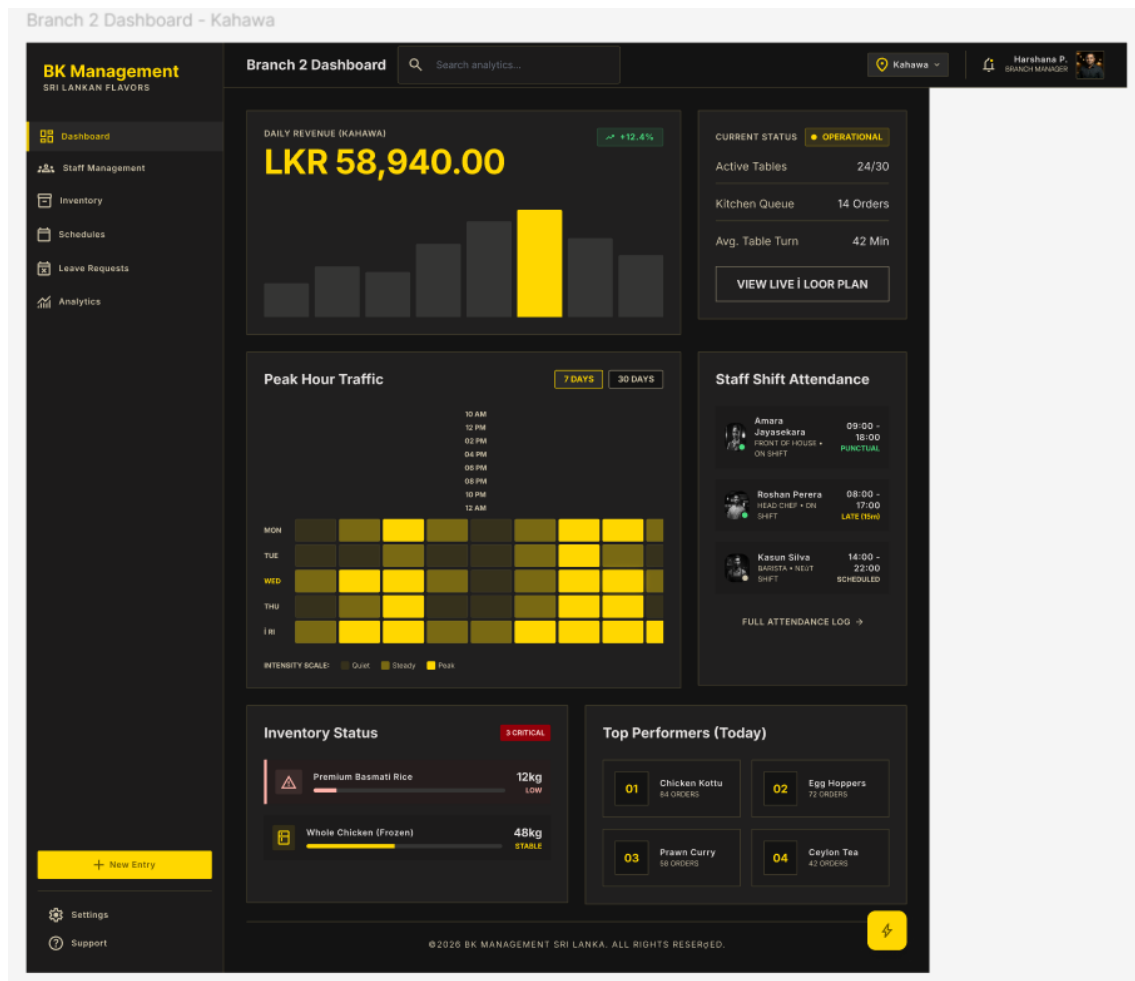


Figure 3.19: UI - Overall Analytics

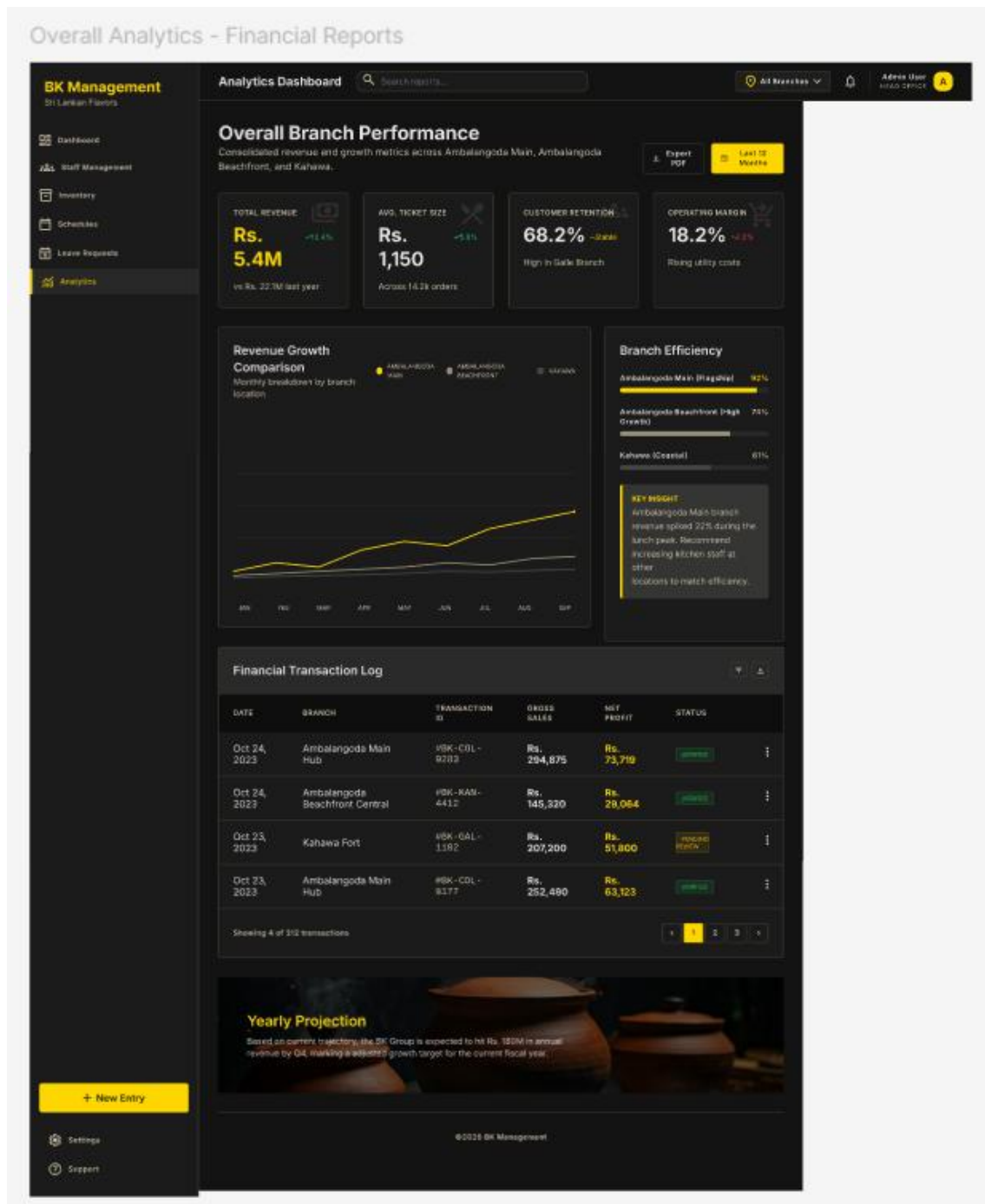
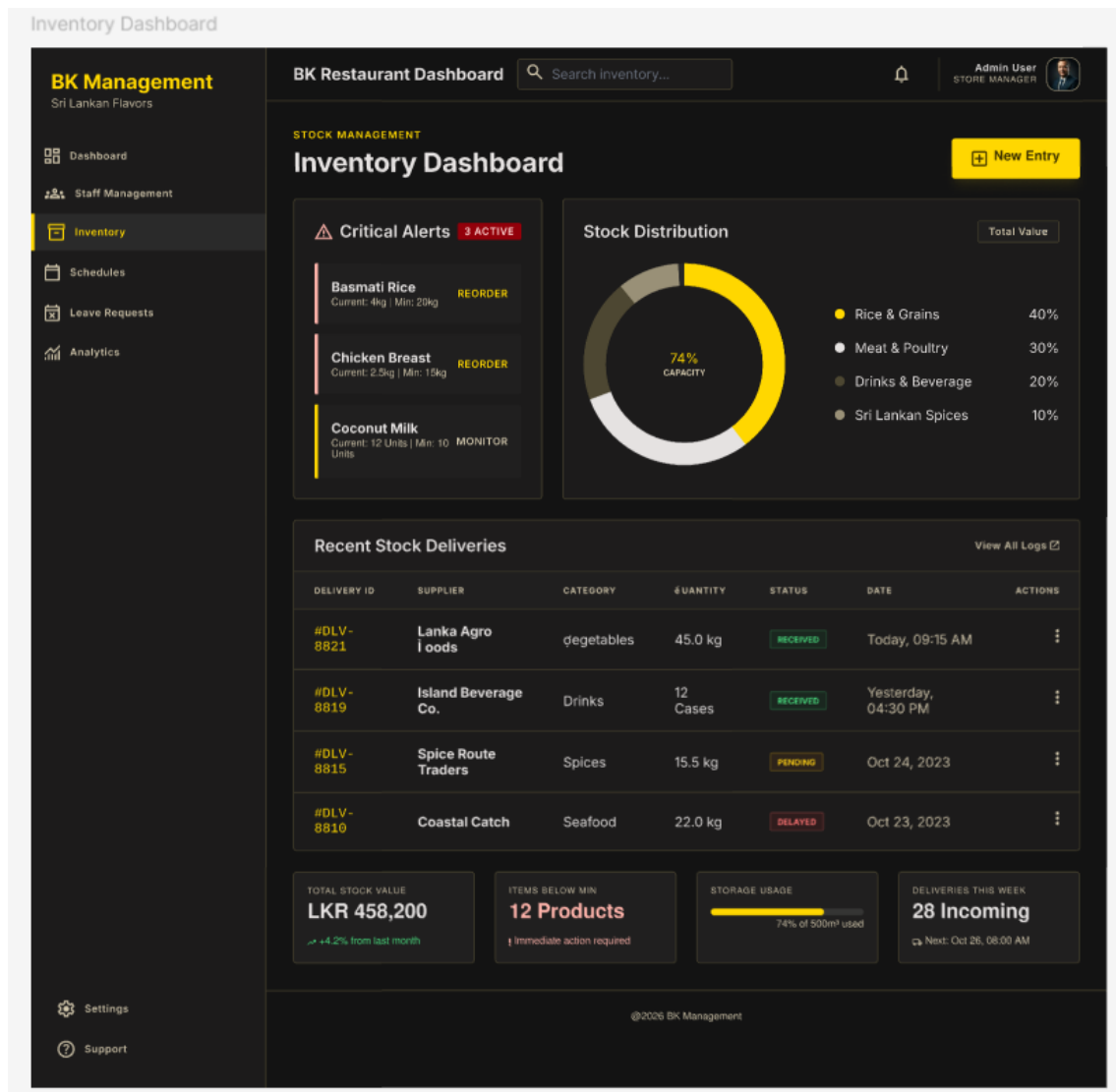


Figure 3.20: UI - Inventory Dashboard



UI - Stock Management

Stock Management & Control

BK Management
Sri Lankan Flavors

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

+ New Entry

Settings

Support

BK Restaurant Dashboard

Search inventory...

Inventory Management

Manage your stock levels, suppliers, and procurement alerts in real-time.

Total Categories
12

Low Stock Alerts
08

Out of Stock
03

Last Reorder Value
LKR 45,200

All Items

Perishables

Dry Goods

Liquids

ITEM NAME	CATEGORY	QTY	UNIT PRICE	STATUS	ACTIONS
Samba Rice (Premium)	Dry Goods	250 kg	LKR 210	IN STOCK	
Organic Tomatoes	Vegetables	5 kg	LKR 450	LOW STOCK	
Ceylon Cinnamon	Spices	15 kg	LKR 1,200	IN STOCK	
Chicken Breast	Meat	0 kg	LKR 1,150	OUT OF STOCK	
Ginger Beer (EGB)	Drinks	48 Units	LKR 180	IN STOCK	

Showing 5 of 124 items

1

2

3

UI - Stock Usage Analytics

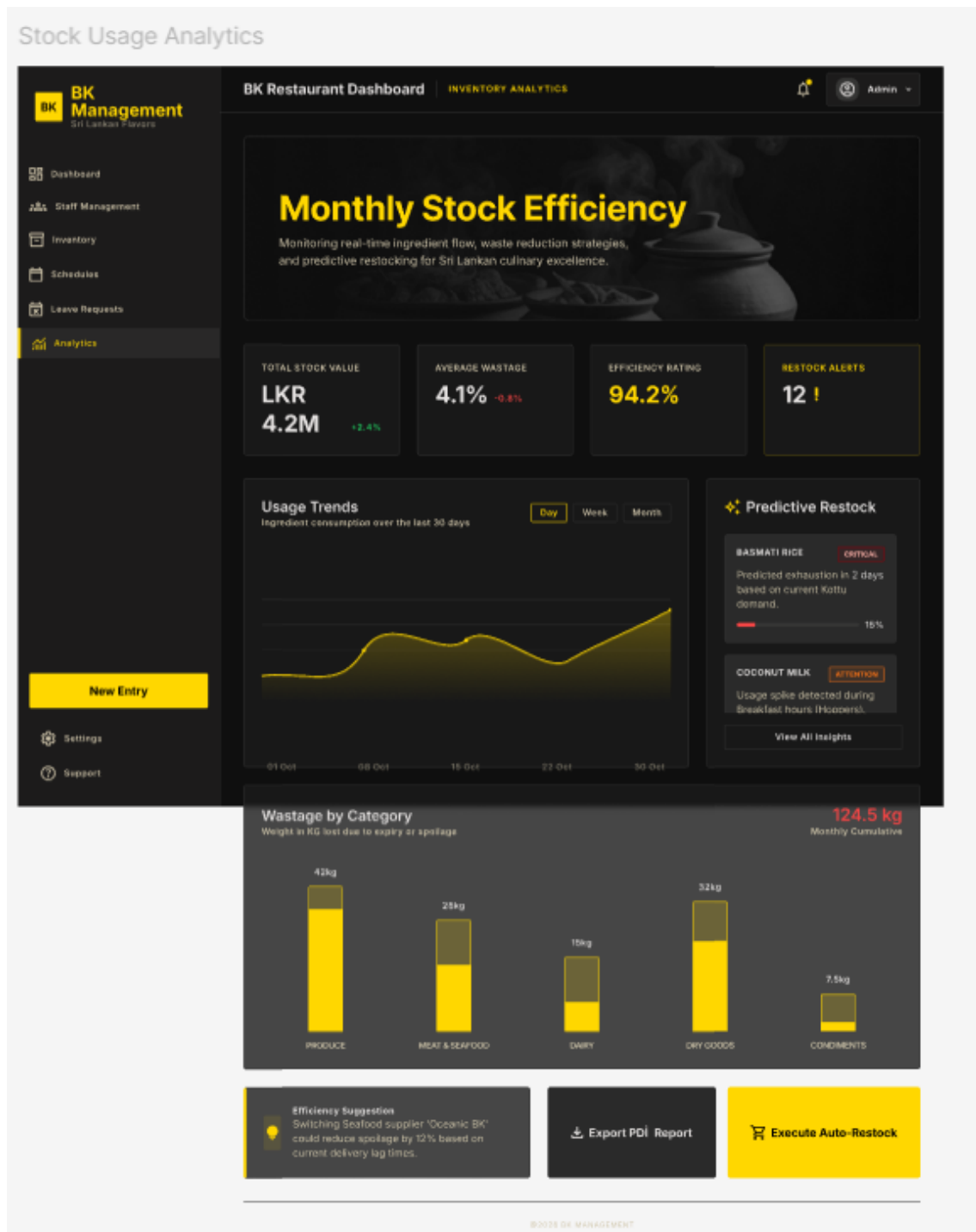
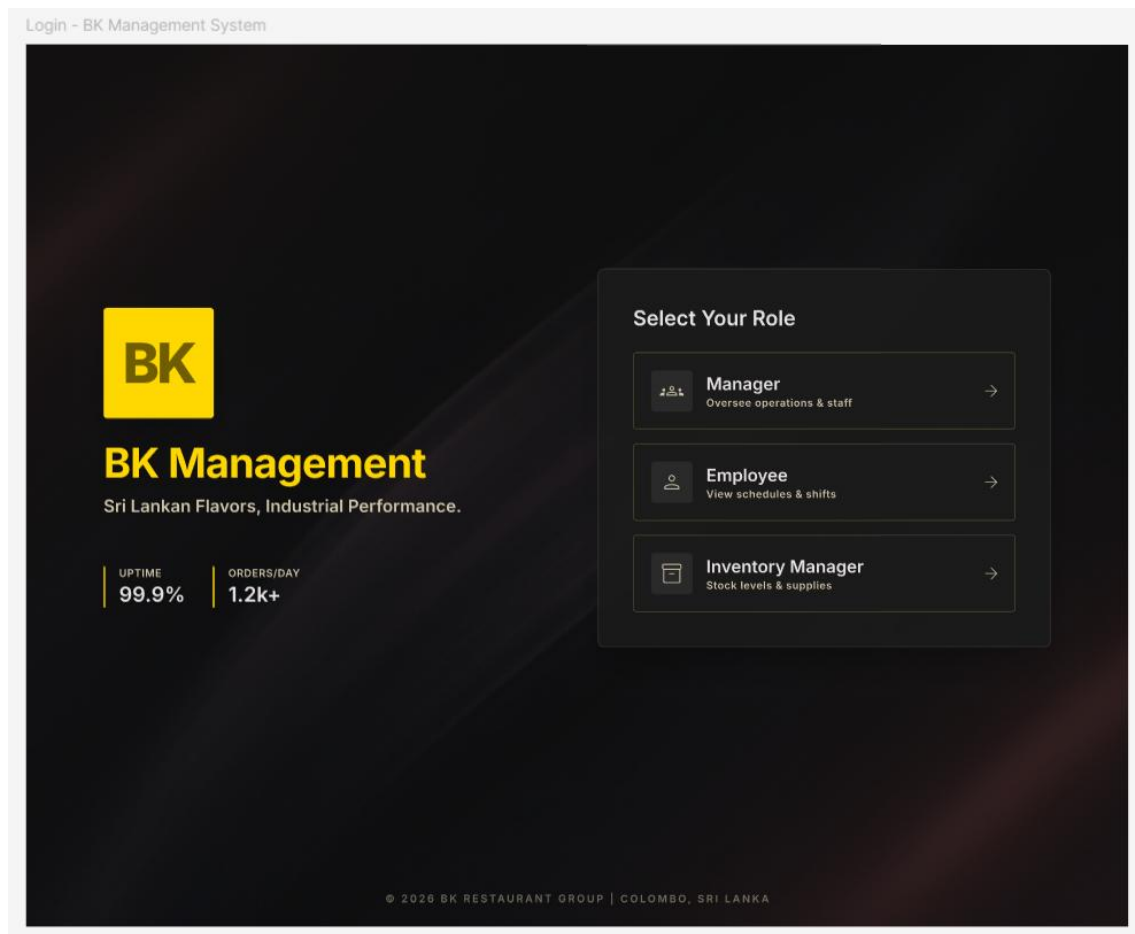


Figure 3.21: UI - Login Screen



UI - User Profile Settings


BK Management
SRI LANKAN FLAVORS

- Dashboard
- Staff Management
- Inventory
- Schedules
- Leave Requests
- Analytics
- Settings**
- Support

BK Restaurant Dashboard

Branch Selector ▾
Logout



Dinesh Wickramasinghe
General Manager

Colombo Main Branch & Head Office
View Public Profile


Contact Information

FULL NAME

EMAIL ADDRESS

PHONE NUMBER

PREFERRED LANGUAGE

English (UK) ▾

RESIDENTIAL ADDRESS

UPDATE PROFILE


Security & Access

Update Password

It's a good practice to change your password every 90 days to maintain high account security.

CHANGE PASSWORD

Two-Factor Authentication

Secure your account with SMS or Authenticator App.

☒

BRANCH ASSIGNMENT


Colombo HQ
72 Galle Road, Colombo 03

Employee ID

BK-MO-0012

Joined Date

Mar 15, 2021

Reporting To

Dinesh Wickramasinghe


PRIMARY PERMISSIONS

☒ FINANCIAL_READ
 ☒ STAFF_WRITE

☒ SYSTEM_ADMIN

LOGIN SESSIONS
REVIEW ALL

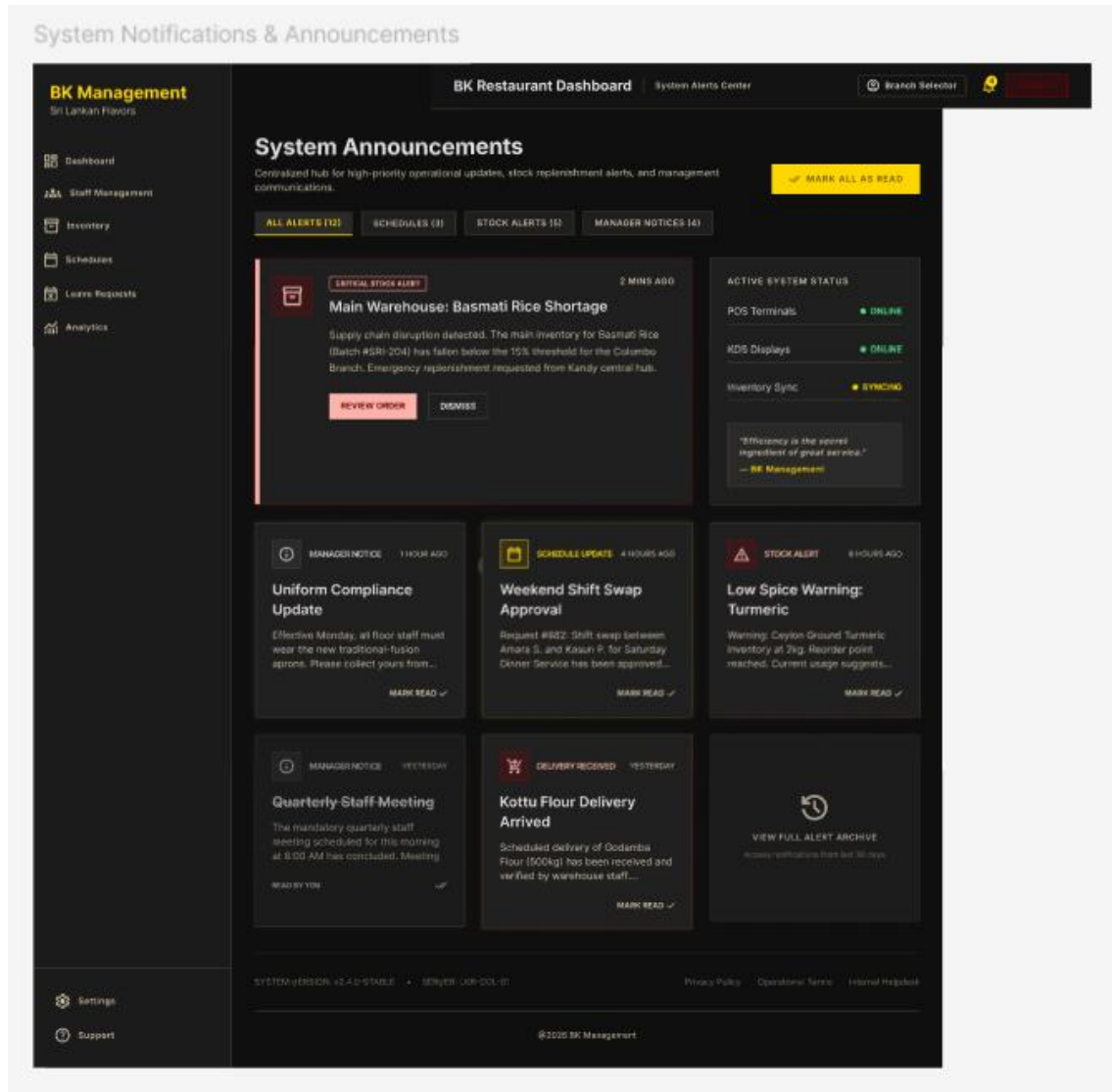

MacBook Pro 16"
Colombo • Current Session


iPhone 15 Pro
Colombo • 2 hours ago


Windows Desktop
Colombo • Yesterday

System Design Project Final Report

UI - System Notifications



3.5 Security Design (OWASP Top 10)

A secure system architecture was prioritized. The following OWASP Top 10 vulnerabilities were mitigated:

A01:2021-Broken Access Control: This was addressed via strict Role-Based Access Control (RBAC) and JWT validation on every API endpoint.

A02:2021-Cryptographic Failures: All data in transit is encrypted via TLS 1.3. User passwords are encrypted at rest using bcrypt with a salt factor of 10.

A03:2021-Injection: This was addressed by using a robust ORM which parameterizes all SQL queries, neutralizing SQL injection vectors.

3.6 Non-Functional Design Decisions Mapping

Non-functional requirements were successfully mapped to specific architectural implementations:

Performance (NFR-01): The requirement for the KDS to sync within 2 seconds was addressed by implementing WebSocket connections (Socket.io).

Reliability (NFR-02): The POS offline-first requirement was addressed using IndexedDB and Service Workers to cache transactions locally during outages.

3.7 UX Decisions and Rationale

To ensure the system is intuitive and rapid for restaurant staff, the user experience was driven by the following key decisions:

Large Touch Targets: The POS and Kitchen Display System (KDS) utilize large, tappable tiles to reduce input errors during fast-paced service, catering to staff with low-to-medium technical proficiency.

Color-Coded Indicators: Status indicators (e.g., green/red for stock levels) allow staff to instantly recognize urgent items at a glance.

Guided Rider Flow: The Delivery Rider App features a step-by-step guided flow with integrated navigation, minimizing decision fatigue while on the road.

Real-time Dashboards: The Owner Dashboard provides high-level visibility with pre-built charts, updating automatically in real-time without requiring manual page refreshes.

3.8 Interface & API Design

The system relies on a combination of internal and external interfaces to facilitate smooth operations:

External Integrations: The platform integrates with the Google Maps Platform for live GPS tracking of delivery riders, thermal receipt printers for physical POS invoicing, and cloud-based Email/SMS Gateways (e.g., SendGrid/Twilio) for automated notifications.

Internal API Structure: Internal communication between the React frontend, Flutter mobile app, and Node.js backend occurs via a versioned RESTful API (/api/v1/...). Payloads are exchanged in JSON format and secured using JWT-based authentication headers. Core backend modules communicate internally through direct service-layer calls.

3.9 Security Design

Robust security measures have been implemented to protect operational data and align with OWASP Top 10 guidelines:

Authentication & Authorization: All access is governed by a centralized Authentication Module utilizing salted bcrypt hashing and JWT tokens. Strict Role-Based Access Control (RBAC) middleware ensures users can only access endpoints and data pertinent to their specific branch and role.

Data Protection: Data in transit is strictly encrypted via HTTPS/TLS, preventing interception. Critical backend configurations and API keys are stored securely as environment variables.

Input Validation & Integrity: All incoming API requests are validated against strict schemas to reject malformed payloads. Status transitions (e.g., order dispatching) follow strict state machines to prevent invalid business operations. Comprehensive audit logs record all sensitive actions for traceability.

CHAPTER 4: CONCLUSION

4.1 Objectives Met

The BK Restaurant Management System was successfully designed to centralize branch operations and eliminate the inefficiencies of manual ledger tracking. The proposed system replaces ad-hoc WhatsApp communication with a robust Point of Sale (POS) and automated Kitchen Display System (KDS), thereby significantly reducing order preparation errors. Furthermore, the integration of a Flutter-based Rider Application

enables real-time GPS tracking of deliveries, fulfilling a critical requirement for operational oversight. By leveraging a centralized PostgreSQL database and React frontend, real-time analytics and inventory alerts are seamlessly provided to the business owner, directly addressing the pain point of sudden stock-outs.

4.2 Limitations

Despite the successful architectural design, certain limitations are acknowledged in the current proposed system. Direct integration with third-party delivery platforms such as UberEats and PickMe was excluded from the initial scope due to API licensing constraints. Consequently, orders originating from these platforms must still be manually entered into the POS by the cashiers. Additionally, the offline caching capability of the POS is limited to transactional data; heavy analytical queries cannot be executed until network connectivity is restored.

4.3 Future Work

Several enhancements are recommended for future iterations of the system. First, the development of a direct webhook integration for third-party food delivery services is proposed to fully automate external order ingestion. Second, a predictive analytics module powered by machine learning could be introduced to forecast inventory depletion rates based on historical seasonal data. Finally, a customer-facing web application could be deployed to allow patrons to place takeaway orders directly, further reducing the dependency on phone calls and WhatsApp messages.

CHAPTER 5: APPENDICES

Appendix A: Software Requirements Specification (SRS)

The complete and detailed Software Requirements Specification (SRS) document, which includes the full feasibility study, business processes, and comprehensive requirement breakdowns, has been attached as an appendix. Please navigate to the Appendix A: SRS bookmark in the PDF viewer to read it.

Appendix B: Software Design Specification (SDS)

The complete and detailed Software Design Specification (SDS) document, which includes the database schema, internal/external API specifications, detailed architecture, and security design, has been attached as an appendix. Please navigate to the Appendix B: SDS bookmark in the PDF viewer to read it.